

长沙学院

CHANGSHA UNIVERSITY

本科生毕业设计说明书

设计题目：基于大语言模型的避难所游戏与智能
NPC 系统的设计与实现

学 院：计算机科学与工程学院

专 业：软件工程

学 生 姓 名：刘禹仝

班 级：22 软件 01 学号 B20220502126

校内指导教师：周景 职称 讲师

校外指导教师： 职称

最终评定成绩：

长沙学院教务处

二〇二六年五月

长沙学院本科生毕业设计说明书

基于大语言模型的避难所游戏与智能 NPC 系统
的设计与实现

学 院： 计算机科学与工程学院

专 业： 软件工程

学 号： B20220502126

学 生 姓 名： 刘禹仝

校内指导教师： 周景 职 称 讲师

20 26 年 5 月

毕业设计原创性声明

本人郑重声明：所呈交的毕业设计是本人在老师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本毕业设计不包含任何其他个人或集体已经发表或撰写的成果作品。对本毕业设计的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果，并承诺因本声明而产生的法律结果由本人承担。

学生签名：_____

日期：2026 年 5 月 22 日

毕业设计版权使用授权书

本毕业设计作者完全了解学校有关保留、使用设计的规定，同意学校保留并向国家有关部门或机构送交毕业设计的复印件和电子版，允许毕业设计被查阅和借阅。本人授权长沙学院可以将本毕业设计的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本毕业设计。

本毕业设计属于

1、保密☐，在____年解密后适用本授权书。

2、不保密☒。

（请在以上相应方框内打“√”）

学 生 签 名：_____

日期：2026 年 5 月 22 日

校内指导教师签名：_____

日期：2026 年 5 月 22 日

校外指导教师签名：_____

日期：2026 年 5 月 22 日

摘 要

本文以本人开发的 Godot 项目《Mirdo》为研究对象，围绕一个地下避难所场景实现生存互动、物资整理、外出探索 and 智能 NPC 表现。项目没有把重点放在实时射击或敌人战斗上，而是把“是否外出、去哪外出、携带什么、回来后状态是否还能维持”作为主要矛盾。客户端使用 Godot 4 和 GDScript 编写，论文中统一用 `res://` 表示客户端资源路径，用 `server://` 表示本地后端路径。

在实现上，玩家通过第一人称控制器在避难所中移动，与食品柜、医疗柜、装备柜、门、物品和 Mirdo 交互。物品、地点规则、外出携带栏、地图进度和存档均尽量采用 `Resource` 或场景节点保存；外出系统提供 12 格携带栏，确认后由

`res://levels/outing/outing_map_level_v3.gd` 汇总 `location`、`loadout`、`time`、`available_loot`、`unlocked_neighbors` 和 `provider` 等字段，向 `server://app/expedition_orchestrator.py` 对应的 `/outing/resolve` 发送请求。后端生成故事、风险、收益和伤害建议后，Godot 再校验掉落路径、数量、伤害范围和失败状态，最后才写回库存、角色状态、时间、地图解锁和存档。

Mirdo 的实现也不是单纯聊天窗口，而是组合式节点 AI。角色场景

`res://characters/mirdo/mirdo_character.tscn` 中拆分了感知、AI 黑板、动作语义、情境行为包、心智状态、自主规划、自主生活、意图解释、动作执行、归来迎接、动画行为树、表情口型、头部注视、玩家感知和 `SpringBone` 二级物理等组件。后端 `/chat` 只返回对白、表情、口型和可执行意图，本地组件再判断目标、动作、姿态和移动是否合法；模型不可用时，Godot 与后端都提供本地 `fallback`。测试与联调结果表明，系统能够跑通“避难所整理—Mirdo 陪伴—外出选择—AI 结算—归来反馈—资源回流—继续生存”的闭环。

关键词：Godot；生存游戏；AI 结算；角色 AI；资源管理

ABSTRACT

This thesis is based on my Godot project Mirdo. The project implements a small shelter survival interaction prototype rather than a real-time shooting game. The main player decisions are whether to leave the shelter, where to go, what to carry, and whether the returned resources and character states are still sustainable. The client is developed with Godot 4 and GDScript. In this thesis, `res://` is used for client resources and `server://` is used for the local backend.

On the client side, the player moves inside the shelter from a first-person perspective and interacts with cabinets, items, the exit door, and Mirdo. Items, location rules, loadout data, map progress, and save data are organized through Resources and scene nodes. The outing map provides a 12-slot loadout. After confirmation, `res://levels/outing/outing_map_level_v3.gd` builds a request containing location, loadout, time, `available_loot`, `unlocked_neighbors`, and provider information, then sends it to `/outing/resolve` handled by `server://app/expedition_orchestrator.py`. The backend returns narrative and resolution suggestions, while the Godot client validates loot paths, amounts, damage values, and failure states before applying inventory changes, character state costs, time progression, map unlocking, and save persistence.

Mirdo is also implemented as a compositional AI character instead of a simple chat box. The character scene contains perception, AI blackboard, action semantics, situation behavior packs, mind state, autonomous planning, autonomous life, intent interpretation, action execution, return greeting, animation behavior tree, facial expression, viseme, head look-at, player awareness, and SpringBone secondary motion components. The backend `/chat` route only provides dialogue, expressions, visemes, and executable intents; local Godot components still validate targets, actions, posture, and movement. When the model is unavailable, both the backend and the client provide deterministic fallbacks. The implemented prototype forms the loop of shelter management, Mirdo companionship, outing selection, AI resolution, return feedback, and resource feedback.

Keywords: Godot; survival game; AI resolution; character AI; resource management

目 录

摘 要	4
ABSTRACT	5
第 1 章 绪论	7
1.1 选题背景与意义	7
1.2 国内外研究现状	8
1.3 主要研究内容	9
1.4 论文组织结构	10
第 2 章 相关技术概述	10
2.1 Godot 4 游戏引擎	10
2.2 GDScript 与组合式节点设计	10
2.3 Resource 数据驱动设计	11
2.4 游戏 AI 与行为规划	11
2.5 FastAPI 后端服务	12
2.6 大模型结构化输出	13
第 3 章 需求分析	13
3.1 游戏目标与用户需求	13
3.2 功能需求分析	13
3.3 非功能需求分析	14
3.4 可行性分析	15
第 4 章 系统总体设计	16
4.1 系统总体架构	16
4.2 游戏核心循环设计	17
4.3 功能模块划分	18
4.4 数据资源设计	19
4.5 客户端与后端交互设计	19
第 5 章 系统详细设计与实现	20
5.1 主菜单与存档槽设计	20
5.2 第一人称避难所交互实现	21
5.3 组合式节点架构实现	22
5.4 物品、背包与庇护所库存实现	23
5.5 角色状态与游戏时间系统实现	25
5.6 外出地图与地点解锁实现	25

5.7 AI 外出结算系统设计与实现	26
5.8 Mirdo 角色 AI 行为系统设计与实现	29
5.9 Mirdo 角色表现与 SpringBone 物理模拟实现	37
5.10 后端 AI 服务设计与实现	40
第 6 章 系统测试	44
6.1 测试环境	44
6.2 功能测试	44
6.4 存档与持久化测试	46
6.5 测试结果分析	46
结 论	46
参考文献	47
附 录	48
致 谢	49

第 1 章 绪论

1.1 选题背景与意义

做这个课题时，一个很直接的感受是：现在个人开发者已经可以借助 Godot 这类引擎，把一个可运行的游戏原型做得比较完整。游戏项目和普通管理软件不太一样，界面、数据和逻辑只是基础，还要考虑玩家按键后的反馈、场景里的表现以及连续游玩时的节奏。生存类玩法尤其依赖这种节奏，玩家一边消耗食物、水和药品，一边判断是否要冒险外出，长期下来就会形成资源压力和下一轮决策。

在本项目中引入大模型，并不是为了让它接管游戏规则，而是想解决固定事件文本容易重复的问题。外出探索、角色对白和情绪反馈都需要根据当时的物资、地点和角色状态变化，如果全部手写分支，内容量会很快膨胀。大模型适合根据上下文组织一段叙事或给出结构化建议，但它也可能返回格式不完整的 JSON、编造不存在的物品，或者给出与当前状态矛盾的结果。所以项目最终采用的做法是：模型只给出建议和描述，真正改变库存、状态、时间和存档的操作仍由 Godot 客户端检查后执行。

《Mirdo》正是在这一背景下设计的避难所生存互动游戏。项目选择 Godot 4 作为开发引擎，使用 GDScript 组织客户端逻辑，并通过本地 Python FastAPI 后端连接 OpenAI-compatible 模型。游戏以地下避难所为核心据点，玩家需要整理物资、照顾 Mirdo、查看状态，并在资源不足时外出探索。与传统 FPS 战斗游戏不同，本项目当前

没有完整实时战斗系统，而是将武器、工具、医疗品等作为外出风险结算和资源管理的一部分。系统重点在于构建“避难所管理—外出探索—AI 结算—资源回流”的循环，并让 Mirdo 角色具备一定的自主行为和智能陪伴表现。

这个课题对我来说主要有三点价值。第一，它把“活下去”的资源压力和 Mirdo 的陪伴反馈放在同一个避难所空间里，玩家不是单纯点菜单，而是在柜子、角色状态和外出入口之间来回判断。第二，项目开发中大量使用 Resource、场景树和组件节点，物品、地点、存档和 AI 行为都尽量拆开配置，后续加一个新物品或新地点时不需要改很多控制脚本。第三，大模型只参与叙事、对白和意图建议，最终写入库存和状态前还要经过客户端校验，这一点比单纯接一个聊天接口更符合游戏项目的稳定性要求。

在实际开发过程中，项目经历了从“第一人称避难所交互”到“外出地图结算”，再到“Mirdo 自主行为”的逐步扩展。早期如果把所有逻辑都写进控制器脚本，物品、存档、AI、UI 很快会互相耦合；因此后续改为组合式节点：能挂在场景里的能力尽量作为组件节点，能配置的数据尽量放进 Resource，跨场景状态才放到 Global 或 SaveManager。论文后文对系统的描述也按照这个实现顺序展开，而不是抽象地套用一個通用游戏框架。

1.2 国内外研究现状

从游戏开发技术看，现代游戏引擎通常提供场景组织、脚本系统、资源管理、动画系统、物理系统和 UI 系统。Godot 引擎采用节点与场景的组合方式组织游戏对象，并提供 GDScript、C# 等脚本语言。其官方文档指出，Godot 的游戏通常由场景树中的节点组成，开发者可以通过复用场景和节点构建复杂项目^{[1][2]}。Resource 资源机制也使 Godot 适合保存物品、规则、配置和存档等数据^[4]。

从游戏 AI 研究看，游戏中的 NPC 行为常用有限状态机、行为树、效用 AI、规划系统和黑板模型等方法实现。Yannakakis 和 Togelius 将游戏 AI 的研究范围扩展到角色行为、交互叙事、程序化内容生成和玩家建模等方向^[10]。Millington 和 Funge 也强调游戏 AI 的目标不是追求通用人工智能，而是在游戏约束下实现可信、可控和高效的行为表现^[11]。Buckland 对状态机、感知、路径与目标评价等实用游戏 AI 方法进行了系统整理^[17]，Mark 则从效用曲线、权重和决策评分角度说明了游戏角色行为选择的数学基础^[18]。黑板架构强调多个知识源通过共享工作区协作解决问题，Hearsay-II 是该思想的经典系统案例^[22]，Hayes-Roth 进一步从控制问题角度总结了黑板控制架构^[8]；行为树因结构清晰、层次化和可复用性被广泛用于游戏 NPC 和机器人控制^{[12][20]}；Orkin 关于实时游戏规划的研究说明，目标和动作组成的规划结构有利于 NPC 应对脚本或有

限状态机难以覆盖的变化情境^[19]。通用人工智能教材也将智能体、搜索、规划和不确定环境下的决策作为 AI 系统的基础问题^[21]，这些思想为本文将 Mirdo 拆分为感知、状态、目标、决策和执行层提供了通用理论背景。

已有研究说明，大模型可以参与角色对话、行为描述和代理模拟。Park 等人的 generative agents 把记忆、反思和计划放在一起，用来模拟较可信的日常行为^[13]；ReAct 关注语言模型在推理后选择行动的过程^[14]；RAG 则通过检索外部资料来减少生成内容的漂移^[15]。这些工作对本文有启发，但游戏运行时还有帧率、接口延迟、存档安全和动作可执行性等限制。因此，Mirdo 的日常移动、看向、检查柜子和坐下休息主要由本地组件完成，大模型只在玩家自然语言交互、较复杂的意图解释和外出叙事结算中参与，最后仍回到客户端校验。

具体到 Mirdo 的 AI 行为架构，本文并没有照搬单一算法，而是将上述研究转化为适合 Godot 项目的组合式节点实现：CharacterAIBlackboardComponent 对应黑板架构，用于汇总 perception、known_nav_points、mind_state、resource_stats、player_awareness、current_behavior 和 action_contract；CharacterAutonomousPlannerComponent 借鉴效用 AI 思想，根据语义标签、心智变量、距离和冷却时间打分^[18]；CharacterActionSemanticsComponent 与 CharacterAnimationBehaviorTreeComponent 借鉴行为树的层次化和动作可控思想，把动作分成 idle、locomotion、posture、work、react 等语义域^{[12][20]}；AIDialogueComponent 与后端 CharacterBehaviorPlanner 则把大语言模型限制在对白、情绪和高层意图建议层，使生成式 AI 能增强陪伴感，但不直接越过客户端校验。

1.3 主要研究内容

围绕《Mirdo》的实际开发过程，本文主要说明以下几个部分：

第一，避难所内的第一人称交互。项目使用 Godot 4 搭建 3D 避难所，玩家可以移动、转向、查看柜子、打开物品面板，也可以通过场景里的入口进入外出地图，并和 Mirdo 产生对话或交互。

第二，资源和状态相关系统。物品数据、柜子库存、庇护所汇总库存、角色饥饿口渴、时间推进和存档都以 Resource 或组件脚本实现，这些内容共同决定玩家是否需要外出、外出后能否安全回到避难所。

第三，外出地图和 AI 结算。外出部分没有做成实时战斗，而是让玩家在 2D 地图上选择地点、配置 12 格携带栏，再由后端根据地点规则、携带物和可掉落物生成一次探索结果；Godot 端只接受通过校验的资源和状态变化。

第四，Mirdo 的角色行为。Mirdo 不是只有聊天窗口，而是由感知、黑板、心智状态、情境行为包、动作语义、执行器、表情口型和归来事件等节点共同驱动。她会根据玩家距离、柜子标签、疲劳或好奇状态做出看向、检查、坐下、起身、欢迎归来等低频行为。

第五，后端服务。后端用 FastAPI 提供本地接口，Godot 主要访问 /health、/model/probe、/chat 和 /outing/resolve。对话接口返回后还会经过 CharacterBehaviorPlanner 做动作和表情整理，长期记忆写入 SQLite，并配合 MemoryRAGRetriever 与 Chroma RAG 检索历史内容和世界知识。

1.4 论文组织结构

本文共分为六章。第 1 章介绍课题背景、研究意义、国内外研究现状和主要研究内容。第 2 章介绍项目所使用的相关技术，包括 Godot、GDScript、Resource、游戏 AI、FastAPI 和结构化 AI 输出。第 3 章进行需求分析，明确功能需求、非功能需求和可行性。第 4 章说明系统总体架构、核心循环、模块划分和客户端后端交互设计。第 5 章详细介绍各主要功能模块的设计与实现，包括存档、库存、状态、外出地图、AI 结算、Mirdo AI 行为和后端服务。第 6 章对系统功能和接口进行测试，并分析测试结果。最后给出总结与展望。

第 2 章 相关技术概述

2.1 Godot 4 游戏引擎

Godot 是一款开源、跨平台游戏引擎，支持 2D 和 3D 游戏开发。Godot 的核心组织方式是节点和场景。节点是功能单元，场景是节点树的组合，场景可以被复用、实例化和嵌套^[2]。这种机制适合本项目的开发需求：避难所、外出地图、Mirdo 角色、柜子、物品面板和 UI 都可以作为独立场景或节点组件进行组织。

本项目中，主菜单场景位于 `res://levels/menu/MainMenu.tscn`，外出地图场景位于 `res://levels/outing/OutingMap.tscn`，Mirdo 角色场景位于 `res://characters/mirdo/mirdo_character.tscn`。通过 Godot 的场景实例化机制，项目将角色、道具、交互面板和关卡结构拆分为可维护的模块。

2.2 GDScript 与组合式节点设计

GDScript 是 Godot 面向场景架构设计的脚本语言，语法接近 Python，并与节点、

信号、资源和编辑器属性紧密集成^[3]。本项目采用 GDScript 编写主要逻辑，包括玩家控制、库存管理、外出结算、AI 行为、存档和 UI 控制。

本项目的一个重要工程特点是采用组合式节点设计。组合式节点设计不是把一个角色、一个系统或一个界面全部写进单个脚本，而是把对象拆分为多个职责明确的子节点和组件节点，再通过导出变量、NodePath、信号和统一接口进行组合。Godot 的场景树天然适合这种方式：父节点负责组织生命周期和对外入口，子节点负责具体功能，资源文件负责数据配置。这种方式与游戏开发中常见的 Component Pattern 思想一致，即把对象能力拆分为相对独立的组件，避免单个类承担过多职责^[23]。

以 Mirdo 角色为例，角色根节点只承担角色实体和场景组织职责，具体 AI 能力分布在 Components 节点下：状态组件负责生命、饥饿、口渴、精力和心情；感知组件负责读取场景语义对象和导航点；自主规划组件负责根据内部状态与环境对象打分；动作执行组件负责移动、看向、坐下、播放动作；对话组件负责向后端发送上下文并接收结构化结果；表情和口型组件负责面部表现。这样的拆分使 Mirdo 不是一个“巨型 NPC 脚本”，而是由多个可替换、可测试、可扩展的节点组合成智能 NPC。

避难所交互、库存面板和外出地图也采用类似思路。柜子对象由可交互组件、库存资源和 UI 面板组合而成；外出地图由地点标记、详情卡、准备面板、AI 请求节点和结果面板组合而成；后端 AI 结算只作为外部服务接入，客户端仍由节点组件完成校验和写回。组合式节点设计降低了系统耦合度，使后续增加新柜子、新物品、新地点或新 AI 行为时，主要通过增加资源和组件配置完成，而不需要重写整体控制逻辑。

2.3 Resource 数据驱动设计

Godot 中 Resource 用于保存和加载数据资源。官方文档指出，Godot 保存或从磁盘加载的许多对象都属于 Resource^[4]。本项目利用 Resource 实现数据驱动：物品数据、库存数据、外出地点规则、地图进度和存档均使用 .tres 或 Resource 脚本进行表示。

例如，res://scripts/Inventory/ItemData.gd 定义物品名称、描述、图标、最大堆叠数、消耗效果和外出分类；outing_location_rule_resource.gd 定义外出地点的名称、描述、威胁等级、耗时、推荐工具、掉落倾向和 AI 探索规则；save_game.gd 定义存档资源，保存当前场景、玩家数据、世界对象数据和全局数据。Resource 数据驱动的优势在于可以减少硬编码，使设计者能够通过资源文件扩展游戏内容。

2.4 游戏 AI 与行为规划

游戏 AI 的目标是在特定游戏情境中产生可信、可控和有效的行为表现。常见方法

包括有限状态机、行为树、效用 AI、规划系统和黑板架构。黑板架构适合在多个模块之间共享运行时上下文，并通过控制机制决定当前应优先处理的知识源或动作^{[8][22]}；行为树通过层级结构组织条件与动作，适合表达复杂但可维护的 NPC 行为^{[12][20]}；效用 AI 则通过评分选择当前最合适的行为，适合在多个候选行动之间做权衡^{[10][11][18]}；实时规划思想强调通过目标与动作结构支持 NPC 的动态响应，对本项目的“命令解释—目标修复—动作执行”也有参考意义^[19]。

本项目 Mirdo 角色 AI 采用轻量效用打分思想。

CharacterAutonomousPlannerComponent 根据内部状态、目标标签、距离、重复惩罚和冷却时间对候选行为评分，再选择检查柜子、看向玩家、环顾、坐下或执行小动作等行为。**CharacterAIBlackboardComponent** 负责把感知结果、导航点、玩家注视、资源状态、当前任务和动作合同汇总为统一 snapshot，解决“对话组件、规划组件、执行组件各自看到的信息不一致”的问题。动作语义和调度层则吸收行为树的层次化思想，把坐下、起身、移动、工作、反应等动作先放入可控分类，再根据角色姿态和任务优先级执行。大模型不负责每帧控制，而是在玩家对话、复杂意图理解和外出叙事结算时提供结构化建议。

因此在 Mirdo 的实现中，AI 行为架构可以理解为“本地 Agent 骨架 + 大模型语言增强”。本地 Agent 骨架包括感知、黑板、心智状态、情境行为包、效用打分、动作语义、任务调度、导航执行和表现反馈；大模型语言增强包括角色对白、情绪表达、玩家自然语言命令解释、外出探索故事和 RAG 记忆检索。这样的设计与生成式代理中“记忆、计划、行动”的思想相呼应^[13]，也借鉴 ReAct 的“先理解再行动”思路^[14]，但在游戏实现上增加了动作白名单、场景目标校验和客户端状态写回，避免模型生成内容直接破坏存档、库存或角色动作。

2.5 FastAPI 后端服务

FastAPI 是 Python Web 框架，常用于构建 HTTP API 服务。它支持基于类型声明的数据模型、自动文档和异步请求处理，适合为 Godot 客户端提供轻量本地服务^[7]。Pydantic 提供数据模型和校验能力，适合约束后端请求与响应字段^[16]。本项目后端位于 server://，默认监听 127.0.0.1:5678，提供 /health、GET /model/probe、/chat、/outing/resolve 等接口。

Godot 客户端通过 HTTPRequest 或 AI 管理组件访问后端。后端不直接控制游戏对象，而是返回结构化 JSON。客户端根据 JSON 内容执行实际游戏状态变更，从而保持前后端边界清晰。

2.6 大模型结构化输出

^[9]游戏客户端不能只接收一段自然语言，因为库存增减、受伤数值和地图解锁都需要明确字段。OpenAI 关于 Structured Outputs 的文档说明，按 schema 约束输出能提高结构一致性[9]。结合这一思路，本项目后端把模型回复限制为固定 JSON 结构：外出结算关注 title、story、risk_result、loot、health_damage 等字段；角色对话关注 dialogue、expression、action、command、visemes 等字段。这样 Godot 端才能逐项校验，而不是从自由文本里猜测结果。

为了进一步保证安全，项目不是完全信任模型结果。外出系统会过滤掉不在 available_loot 中的资源路径；后端连接失败时客户端会使用本地保守结算；后端已连接但模型失败时，本次外出不会扣除物资或保存地图进展。

第3章 需求分析

3.1 游戏目标与用户需求

《Mirdo》的目标是实现一款以避难所为核心据点的第一人称生存互动游戏。玩家在封闭的避难所中管理物资，查看自身与 Mirdo 的状态，并在资源不足或需要推进探索时外出寻找补给。游戏通过资源消耗、时间推进和外出风险形成决策压力，通过 Mirdo 的角色行为和 AI 对话增强陪伴感。

结合当前版本的实现范围，用户需求还有两个边界需要说明。第一，玩家看到的“风险、丧尸、武器”主要体现在外出结算文本、物品损耗和状态变化中，当前版本不提供完整敌人寻路、命中判定和实时战斗 UI。第二，AI 不是自动替玩家玩游戏；外出 AI 只生成结算建议，Mirdo 对话 AI 只生成对白和动作意图，真正影响存档的数据仍由客户端校验后写入。这个边界是系统设计时反复保留的约束。

用户需求主要包括：能够方便地进入游戏、保存和读取进度；能够在避难所中进行第一人称移动和交互；能够查看和整理物资；能够使用食物、水和医疗品；能够选择外出地点并获得结算结果；能够看到 Mirdo 的对话、动作和表情反馈。

3.2 功能需求分析

系统功能需求如下表所示。

编号	功能名称	需求说明
F1	主菜单与进度槽	支持新游戏、继续游

		戏、进度槽管理、设置和退出
F2	第一人称控制	支持移动、冲刺、蹲伏、跳跃、视角控制和交互检测
F3	场景交互	支持与柜子、门、角色和可拾取物交互
F4	物品与库存	支持物品堆叠、分类限制、跨库存转移、外出携带和使用
F5	角色状态	支持生命、饥饿、口渴、精力、心情、好感等状态
F6	时间系统	支持游戏时间推进、日切和状态衰减
F7	外出地图	支持地点选择、详情查看、准备面板、地点解锁和结果展示
F8	AI 外出结算	支持向后端发送外出上下文并获取故事、风险和收益
F9	Mirdo AI 行为	支持感知、规划、导航、动作、表情、口型和对话
F10	后端服务	支持健康检查、模型探测、对话和外出结算接口
F11	存档系统	支持保存和读取场景、库存、状态、时间和地图进度

3.3 非功能需求分析

系统还需要满足以下非功能需求。

(1) 稳定性。AI 服务不可用或模型失败时，系统不能破坏玩家存档或造成资源异常损失。外出系统需要区分后端不可连接和模型返回失败两类情况，并采用不同处理策

略。

（2）可扩展性。物品、库存、外出地点和 AI 导航点应尽量通过 Resource 或场景语义组件配置，便于后续新增内容。

（3）可维护性。系统应采用组件化设计，避免将库存、AI、动作、UI 和存档全部写在单个脚本中。

（4）可交互性。玩家应能清晰看到物品、状态、外出结果和 Mirdo 的反馈。Mirdo 的 AI 行为应以低频自然表现为主，不能频繁打断玩家。

（5）安全性。大模型只提供结构化建议，不直接修改客户端数据。客户端负责校验资源路径、物品数量和状态变化。

3.4 可行性分析

技术可行性方面，Godot 4 提供 3D 场景、Control UI、AnimationTree、HTTPRequest、Resource 和 GDScript 等功能，能够支持项目所需的客户端系统。Python FastAPI 能够快速构建本地 HTTP 服务，并通过 Pydantic 定义请求和响应模型，适合承担 AI 后端接口。

经济可行性方面，Godot 是开源引擎，项目主要使用本地资源和开源技术实现，开发成本较低。大模型服务可以通过可配置的 OpenAI-compatible 接口接入，既可以使用云端模型，也可以接入本地兼容服务。

操作可行性方面，游戏采用主菜单、进度槽、第一人称移动、柜子面板和外出地图等常见交互形式，用户学习成本较低。外出流程通过地点详情和准备面板引导玩家完成决策。

第 4 章 系统总体设计

4.1 系统总体架构

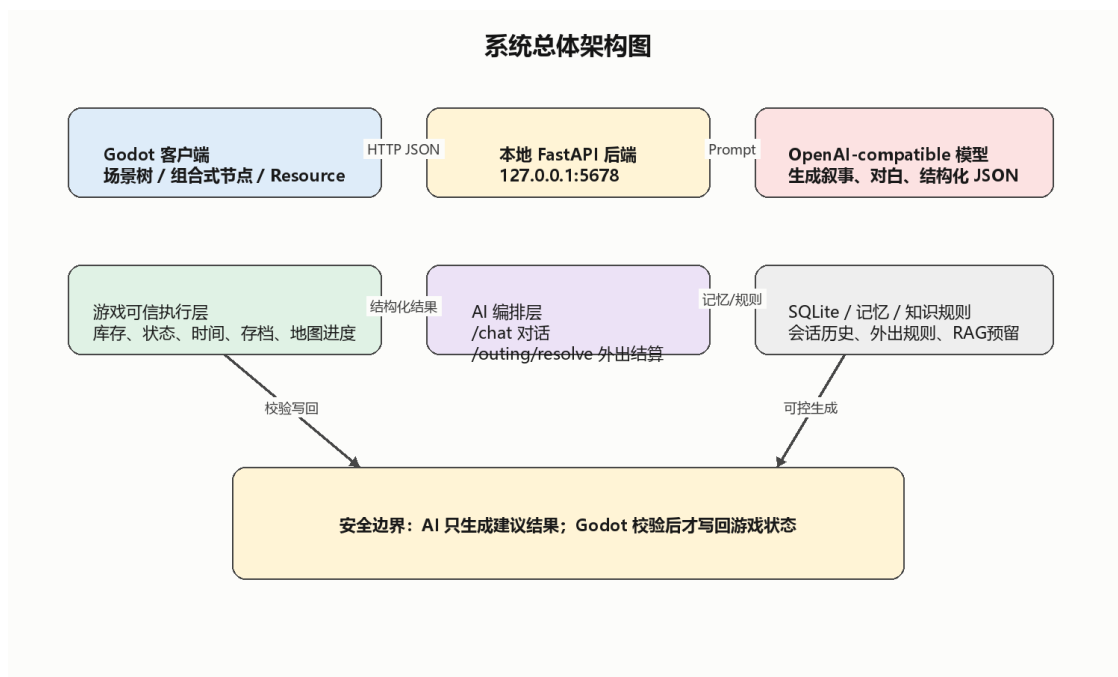


图 4.1 系统总体架构图

《Mirdo》系统由 Godot 客户端和本地 AI 后端两部分组成。Godot 客户端负责游戏场景、玩家控制、库存、状态、时间、外出地图、Mirdo 行为和存档。后端服务负责对话、模型探测、外出 AI 结算、记忆和知识检索等功能。

系统采用客户端可信写回原则。后端可以生成文本和建议结果，但实际的物品扣除、收益入库、状态变化、时间推进和地图解锁都由 Godot 客户端执行。

4.2 游戏核心循环设计

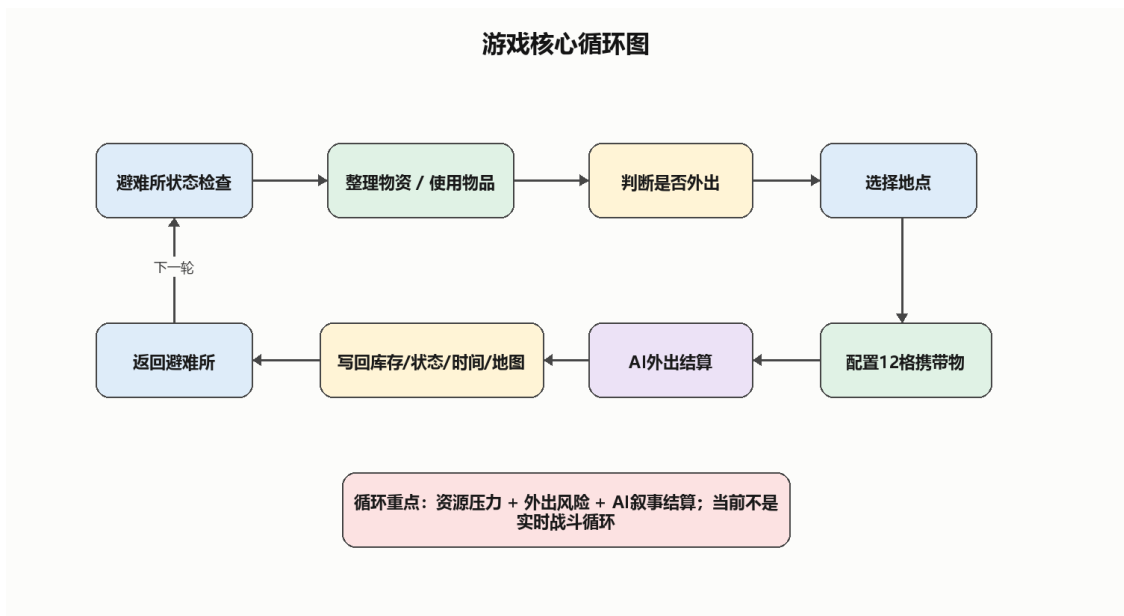


图 4.2 游戏核心循环图

游戏核心循环如下：

其中 Mirdo 自主陪伴不是附属装饰，而是避难所阶段的重要反馈层。玩家整理柜子、查看状态或靠近角色时，Mirdo 会根据场景语义、玩家距离、资源状态和内部心智状态做出低频行为，如看向玩家、挥手、检查食品柜、在医疗柜前查看架子、到工作台前解释、坐下休息或因注视而歪头。这些行为穿插在资源管理过程中，使避难所不只是一个静态菜单，而是具有角色存在感的空间。

以一次典型流程为例，玩家先在避难所检查食品柜和医疗柜，如果水或食物不足，就从大门进入 **OutingMap**；在地图上选择威胁等级不同的地点，把药品、工具或武器放入 12 格携带栏；确认后进入“出发—搜索—返程—结果页”的结算过程。若后端正常返回，结果页展示故事、获得物资、受伤和解锁信息；若后端不可用，则使用本地保守结算；若模型已连接但输出异常，本次行动保持在出发前状态，避免扣物或保存错误进度。这样循环中的每一步都有可回退策略。

在避难所阶段，玩家查看库存和角色状态，使用食物、水、医疗品恢复状态。状态和资源随着时间消耗产生压力。当资源不足时，玩家进入外出地图，选择不同风险和收益倾向的地点，并配置外出携带物。外出完成后，系统根据 AI 结算结果和本地规则写回物资、状态和地图进度，玩家回到避难所继续下一轮决策。

这个循环的重点是资源压力和风险选择，而不是实时战斗。武器、工具和医疗物资

主要作为外出结算中的风险修正因子。

4.3 功能模块划分

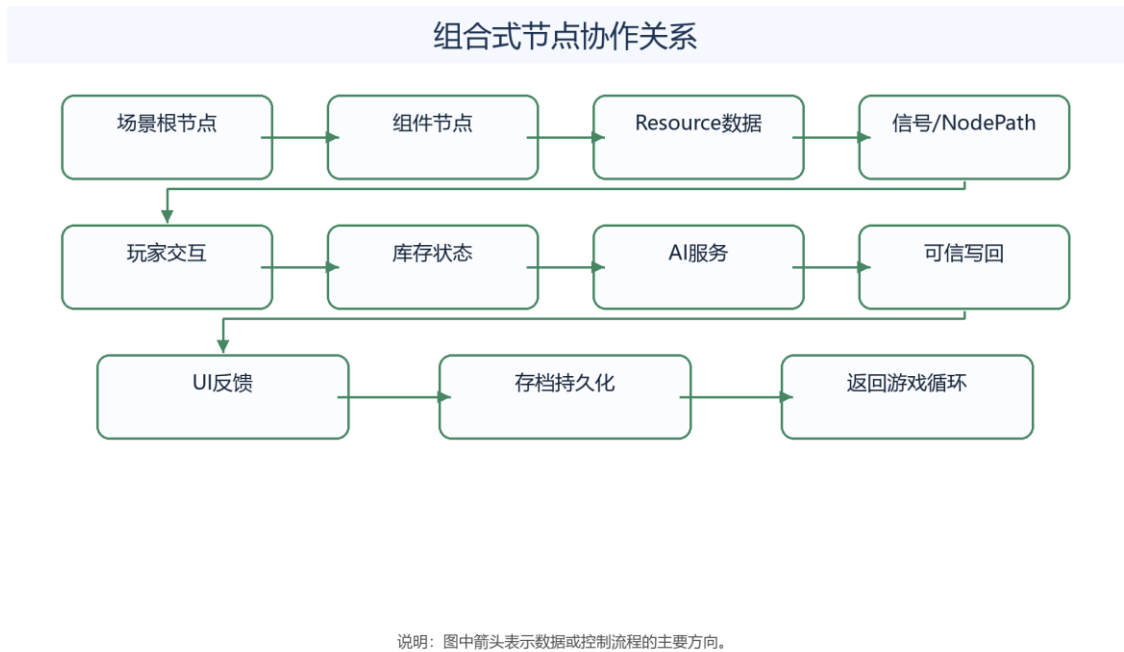


图 4.3 组合式节点协作关系图

按照实际工程文件和运行流程，系统可以拆成以下十个模块。

（1）主菜单与存档槽模块。负责新游戏、继续游戏、进度槽展示、读取、保存和删除。

（2）玩家控制与交互模块。负责第一人称移动、视角控制、交互检测和世界交互面板。

（3）物品与库存模块。负责物品数据、背包、柜子、分类限制、堆叠规则、物品转移和使用。

（4）角色状态与时间模块。负责生命、饥饿、口渴、精力、心情等状态，以及时间推进和状态衰减。

（5）外出地图模块。负责 2D 地图、地点标记、地点详情、准备面板、携带栏、地图解锁和结果页。

（6）AI 外出结算模块。负责构造外出请求、调用后端、解析 AI 结果、失败兜底和客户端写回。

（7）Mirdo AI 行为模块。负责 Mirdo 的感知、心智状态、自主规划、意图解释、导航、动作、表情和口型。

(8) Mirdo 角色表现与物理模拟模块。负责 AnimationTree、RootMotion、BlendShape 表情口型、头部 LookAt、玩家注视反应和 SpringBone 二级物理。

(9) 后端 AI 服务模块。负责 FastAPI 接口、模型配置、对话、外出结算、记忆和知识检索。

(10) 存档模块。负责 Resource 存档、场景恢复、全局库存、时间状态和外出地图进度持久化。

4.4 数据资源设计

项目采用 Resource 数据驱动。主要数据资源包括：

(1) 物品资源。位于 resources/items/*.tres，记录物品名称、描述、图标、最大堆叠数、可用效果、外出分类、AI 规则提示等。

(2) 库存资源。位于 resources/storage/*.tres，记录不同柜子和外出携带栏的默认数据。真实库存保存在各柜子的 InventoryStorageResource 中，ShelterInventoryResource 作为汇总入口。

(3) 地点规则资源。位于 levels/outing/location_rules/*.tres，记录地点 ID、显示名、描述、坐标、威胁等级、耗时、掉落偏向、推荐工具和 AI 探索规则。

(4) 地图解锁资源。位于 levels/outing/unlock_links/*.tres，记录地点之间的解锁关系。

(5) 存档资源。SaveGame 保存当前场景、玩家数据、世界对象数据、全局数据和元数据。全局数据包含庇护所库存、外出地图进度和时间状态。

4.5 客户端与后端交互设计

客户端通过 HTTP 与本地后端通信。AI 服务默认地址为 http://127.0.0.1:5678。Godot 端 AIServiceSupervisor 会检查 /health，若服务未运行则尝试启动 run_server.py。

最新实现中，AISettings 不只保存 base_url、api_key、model 和 proxy_url，还在 Godot 端加入服务健康检查和模型连接测试。客户端先访问 http://127.0.0.1:5678/health 判断本地服务是否可用，再通过 GET /model/probe 让后端使用当前模型配置实际构造 OpenAI-compatible 请求进行探测。AISettingsPanel 和主菜单会显示服务延迟、模型延迟和错误信息，避免玩家进入游戏后才发现模型配置不可用。

主要接口如下：

接口	方法	作用
/health	GET	检查后端服务、模型、

		记忆和知识库状态
/model/probe	GET	测试模型是否可用
/chat	POST	Mirdo 对话和行为建议
/outing/resolve	POST	外出 AI 叙事结算
/session/{id}/history	GET	查询会话历史
/memory/clear	POST	清除记忆

外出结算是本文重点。Godot 端发送地点、携带物、可掉落物、时间和 provider 配置；后端返回故事、风险、收益、线索和伤害；客户端校验后执行状态写回。

第 5 章 系统详细设计与实现

5.1 主菜单与存档槽设计

主菜单场景位于 `res://levels/menu/MainMenu.tscn`，主要入口包括继续游戏、新游戏、游戏进度、设置和退出。进度槽界面位于 `res://controllers/ui/SaveSlotMenu.tscn`，固定展示 `slot_01`、`slot_02`、`slot_03` 三个槽位。玩家可以在槽位中保存、读取或删除进度，删除操作需要二次确认。

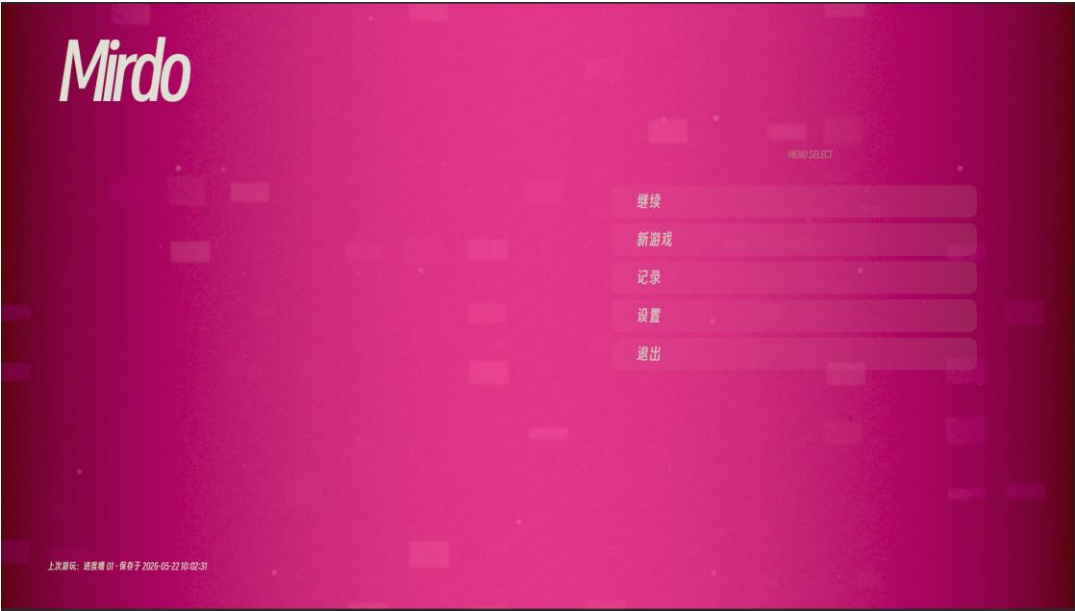


图 5.1 游戏主菜单界面

存档系统使用 Godot Resource 而不是 JSON 文件。SaveGame 资源保存 schema

版本、槽位名、保存时间、当前场景路径、玩家数据、世界对象数据、已销毁对象、全局数据和元数据。SaveManager 通过 ResourceSaver.save() 写入 user://saves/<slot>.tres，并通过安全资源加载器读取存档。

外出地图是独立场景，因此存档系统需要特殊处理。进入外出地图前，Global.go_to_outing_map_from_current_scene() 会保存当前避难所场景并记录返回路径。若在外出地图内触发保存，Global.get_save_scene_path_override() 会将恢复场景覆盖为进入外出前的避难所场景，避免读档后直接回到外出地图。返回避难所时，系统读取进入外出前的存档，再应用外出期间产生的库存、时间和地图进度变化。



图 5.2 存档槽界面

5.2 第一人称避难所交互实现

玩家控制器位于 res://controllers/fps_controller.tscn。其脚本将移动行为拆分为空闲、行走、冲刺、蹲伏、跳跃和下落等状态。玩家通过键盘和鼠标控制角色，在避难所中移动和观察。

交互组件 player_interaction_component.gd 负责检测玩家视线或范围内的可交互对象。当玩家对准柜子、门、Mirdo 或可拾取物时，系统显示交互提示或世界交互面板。

避难所大门挂载外出入口组件，玩家选择“外出”后进入外出地图。床、柜子、医疗柜等对象也通过组件提供各自的交互行为。



图 5.3 Debug 模式下避难所场景与交互感知预览

5.3 组合式节点架构实现

本项目在 Godot 客户端中没有采用单一控制器集中管理所有玩法，而是围绕场景树建立组合式节点架构。组合式节点架构的基本原则是：根节点表示对象身份，组件节点提供功能能力，Resource 保存可配置数据，信号负责低耦合通信，全局单例只承担跨场景状态与流程协调。

在玩家侧，第一人称控制器由状态机节点、相机节点、交互检测组件和 UI 入口组合而成。移动、跳跃、蹲伏、冲刺等行为通过状态节点拆分，交互检测组件只负责发现目标和发出交互请求，不直接处理库存、对话或外出逻辑。这样可以避免玩家控制脚本不断膨胀。

在物品和柜子侧，一个柜子对象通常由场景节点、可交互组件、库存资源和面板控制逻辑组合。柜子本身只表达“这是一个可被打开的世界对象”，具体能够接收哪些物品由库存资源和分类限制决定，物品显示与转移由 UI 面板和转移服务完成。食品柜、医疗柜、装备柜因此可以复用同一套交互方式，只通过资源配置体现差异。

这种拆分也解决了开发中的几个具体问题。外出地图作为独立场景时，不能把返回场景简单保存成当前地图，否则读档后会卡在外出界面；因此 SaveManager 与 Global 需要区分“当前场景”和“外出前避难所场景”。柜子库存也不能只保存 UI 列表，否则重

新进场景后会丢失容器内容，所以 `SaveComponent` 会自动寻找同级库存组件并保存 `inventory_payload`。Mirdo 行为同理，如果后端直接返回节点路径，模型容易产生不存在的目标；改为 `object_id`、`marker role` 和动作白名单后，本地执行层就能稳定判断。

新增的 `CharacterAIBlackboardComponent` 进一步降低了组件之间的直接依赖。它把 `perception`、`known_nav_points`、`mind_state`、`resource_stats`、`player_awareness`、`current_behavior`、`action_contract` 和 `situation_context` 组织成统一 `snapshot`；`AIDialogueComponent` 可把它作为 `/chat` 的 `runtime context` 发送给后端，`AutonomousPlanner` 也可以从黑板中读取情境标签和动作偏置。这样后端、规划器和调试输出看到的是同一份 Mirdo 运行时状态，而不是各自临时拼装的数据。

在 Mirdo 侧，组合式节点设计最为明显。Mirdo 的智能 NPC 能力被拆分为心智状态、环境感知、AI 黑板、动作语义、情境行为包、自主生活、行为规划、意图解释、动作执行、对话、归来迎接、动画行为树、表情口型、自主补给使用、头部注视和玩家感知等组件。每个组件只处理自己的职责：感知组件不直接移动角色，规划组件不直接播放动画，对话组件不直接修改库存，动作执行组件不生成语言内容，黑板组件只汇总上下文而不强制决策。多个组件通过上下文快照、动作合同和信号协同，最终形成避难所内可调试的角色行为链。

在外出地图侧，2D 地图不是单个结算脚本，而是由地点标记、地点规则资源、解锁关系资源、准备面板、12 格携带栏、`HTTPRequest` 节点、结果页和存档写回逻辑组合而成。AI 结算请求节点只负责发送和接收 `/outing/resolve`，而合法性校验、物资写回、时间推进和地图进度保存仍由客户端节点完成。这种拆分保证了大语言模型只参与叙事和建议结算，不会直接控制游戏状态。

通过组合式节点架构，项目形成了较清晰的依赖方向：场景节点组织对象，组件节点完成行为，`Resource` 提供数据，后端 AI 提供结构化生成结果，客户端负责可信写回。该设计符合 Godot 节点场景模型，也使系统更适合毕业设计中的功能说明、模块测试和后续扩展。Mirdo 的组件数量较多，但并不是简单堆叠脚本，而是按感知、决策、执行、表现和物理反馈分层：感知层生成环境快照，黑板层统一上下文，规划层选择行为，调度层解决动作冲突，执行层处理导航和姿态转换，表现层负责动画、表情、口型、头部朝向和 `SpringBone` 二级运动。

5.4 物品、背包与庇护所库存实现

物品系统以 `res://scripts/Inventory/ItemData.gd` 为基础。每个物品资源包含名称、描述、图标、模型、最大堆叠数、是否可使用、生命/饥饿/口渴/精力恢复值、外出分类、

是否可外出携带和 AI 规则提示等字段。外出分类包括 food、medical、material、tool、weapon、special。

库存系统分为玩家背包、柜子库存和外出汇总库存。柜子的真实库存保存在 InventoryStorageResource 中，食品柜只接收食物，医疗柜只接收医疗品，材料箱接收材料，装备柜接收武器、工具和特殊物品。InventoryTransferService 负责跨库存转移，并校验目标容器的分类限制和堆叠上限。

外出准备时，系统不会要求玩家逐个柜子查找物资，而是通过 ShelterInventoryResource 汇总所有可外出携带的物品。需要注意的是，ShelterInventoryResource 不是第二套真实总库存，而是引用各柜子来源的汇总入口。确认外出时，系统仍按来源柜子和槽位扣除物品。

物品使用支持玩家自用和给 Mirdo 使用。玩家直接打开背包时，物品默认作用于玩家。若通过 Mirdo 交互入口打开背包，则背包进入 Mirdo 目标上下文，食物和水等普通可用物品可作用于 Mirdo 的状态组件。医疗类物品当前强制玩家自用，避免 Mirdo 使用医疗品造成规则混乱。

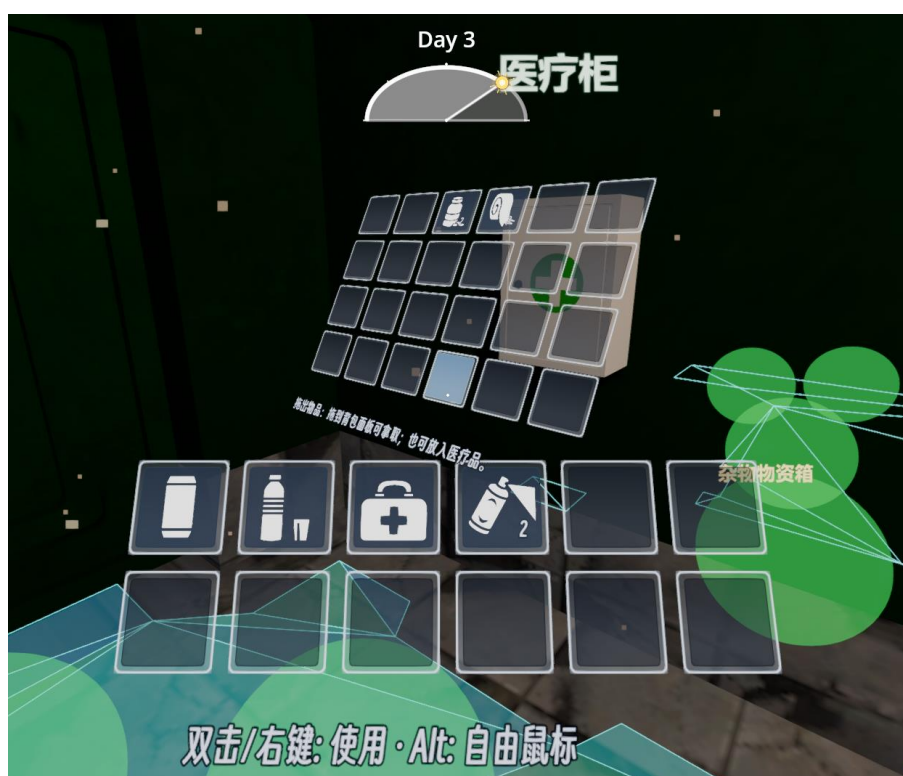


图 5.4 库存与物品槽界面

5.5 角色状态与游戏时间系统实现

角色状态组件 `character_resource_state_component.gd` 维护生命、饥饿、口渴、精力、心情和好感。状态值被限制在 0 到 100 之间，并设置临界阈值。时间推进时，饥饿、口渴和精力会下降；当饥饿或口渴过低时，还会影响生命和心情。

游戏时间由 `game_time_component.gd` 管理，支持 `pass_hours()`、`pass_minutes()`、`skip_sleep()`、`run_stay_home()` 等接口。外出结算后，`Global.advance_outing_time_minutes()` 会推进外出消耗的分钟数，并将时间状态保存到全局存档中。这样即使外出地图本身没有避难所中的时间组件，时间变化也能在返回避难所后恢复。

5.6 外出地图与地点解锁实现

外出地图场景位于 `res://levels/outing/OutingMap.tscn`，主脚本为 `res://levels/outing/outing_map_level_v3.gd`。该场景是独立 2D 地图，包含地图背景、地点标记、地点详情卡、出发准备面板和结果页。

地点规则保存在 `levels/outing/location_rules/*.tres` 中。每个地点资源记录地点 ID、显示名、描述、地图坐标、路线提示、威胁等级、旅行时间、是否初始解锁、掉落偏向、推荐工具、细节提示和 AI 探索规则。AI 探索规则不直接作为普通 UI 文本展示，而是发送给后端用于约束外出故事和结算。

地图进度由 `OutingMapProgressResource` 记录，包括已解锁地点、已发现解锁 key 和各地点成功探索次数。地点之间的解锁关系通过 `unlock_links/*.tres` 描述。当某个地点成功探索次数达到要求时，系统解锁相邻地点。

外出准备面板从庇护所汇总库存读取可携带物资，玩家可配置固定 12 格携带栏。非武器同来源同物品可堆叠，武器一件占一格。携带物将影响 AI 叙事和风险结算。



图 5.5 外出地图与携带栏配置界面

5.7 AI 外出结算系统设计与实现

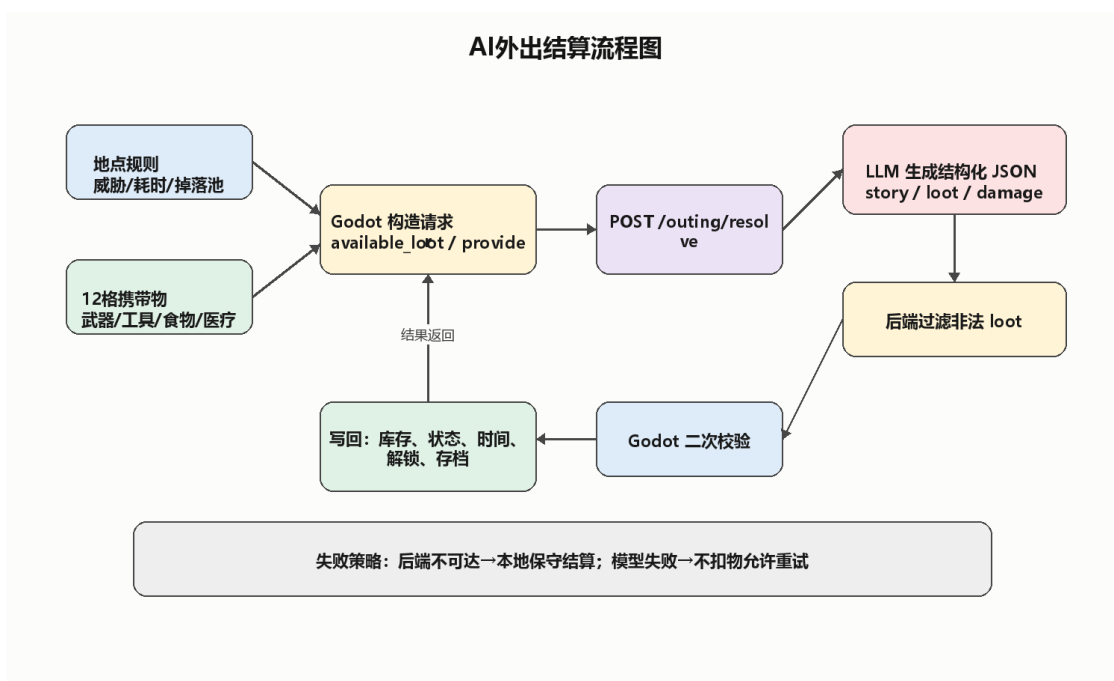


图 5.6 AI 外出结算流程图

AI 外出结算是本项目的重点模块。玩家确认外出后，Godot 端构造请求并发送至后端 /outing/resolve。请求内容包括：

- (1) session_id: 外出会话标识；
- (1) location: 地点 ID、名称、描述、路线提示、威胁等级、掉落偏向、推荐工具

和 AI 探索规则；

(1) **loadout**: 本次携带物品，包括物品 ID、名称、类别、数量、标签和 AI 规则提示；

(1) **time**: 往返时间、搜索时间和总时间；

(1) **available_loot**: 允许 AI 选择的合法掉落物路径；

(1) **unlocked_neighbors**: 本次可能解锁的相邻地点；

(1) **provider**: 模型服务配置。

后端 ExpeditionOrchestrator 根据请求构造提示词，要求模型只输出 JSON，且必须包含 ok、title、summary、story、experience、risk_result、loot、discovered_clues、mood、health_damage 等字段。提示词强调 story 是主要输出，需要按照离开庇护所、靠近地点、搜索、遭遇意外、资源取舍和撤离返程的顺序生成连续叙事。

为了避免 AI 生成不存在的物品，后端会根据 available_loot 建立合法资源路径集合，并过滤掉模型返回中不合法的 item_path。客户端收到结果后还会再次检查资源是否存在，只有合法物品才会写入库存。

结算写回包括以下步骤：

1. 扣除或归还携带物；
2. 判断武器、工具等物品是否损坏；
3. 将 AI 返回的 loot 转换为 Godot 物品资源；
4. 将带回物资写入对应柜子或临时带回包；
5. 根据威胁等级、AI 伤害值和携带物减伤计算状态消耗；

该模块开发时最容易出错的是“AI 说获得了某物，但项目资源里没有这个物品”。为避免这种情况，请求中不只发送地点描述，还发送 available_loot，把允许掉落的资源路径作为候选集合。后端解析结果时先过滤非法 item_path，Godot 收到后再进行第二次资源存在性检查。另一个问题是外出地图和避难所库存属于不同场景，如果在外出场景直接改临时数据，返回后容易丢失；因此结算写回采用增量思路，把获得物资、耗时、解锁地点和状态消耗统一应用到全局存档和庇护所库存。

外出完成后还新增了“真实外出归来事件”。outing_map_level_v3.gd 在非 bunker 地点、总耗时大于 0 的真实外出结算后，会调用 Global.record_real_outing_completed，记录 location_id、location_name、route_minutes、search_minutes、loot_added、carried_count、returned_count、consumed_count、health_damage 和 risk 等字段。Global 将该事件随存档保存，并在返回避难所后通过 player_returned_from_real_outing 信号交给 Mirdo。这样 Mirdo 的欢迎行为只会发生在真实外出回来后，而不会把普通场景切换误认为玩家探险归来。

- 6. 推进外出耗时；
- 7. 记录地点探索次数并解锁新地点；
- 8. 保存进度并显示结果页。

外出结算失败时没有简单地“一律扣物”或“一律成功”。如果本地后端根本连不上，Godot 会走保守的本地结算，让玩家至少能完成一次外出；如果后端已经连上但模型输出坏了，比如 JSON 缺字段或物品路径不合法，则本次行动保持在出发前状态，不扣携带物、不写入战利品、不推进时间，也不保存地图解锁。这个处理是为了避免模型异常把玩家存档改坏。



图 5.7 AI 外出结算结果页

5.8 Mirdo 角色 AI 行为系统设计与实现

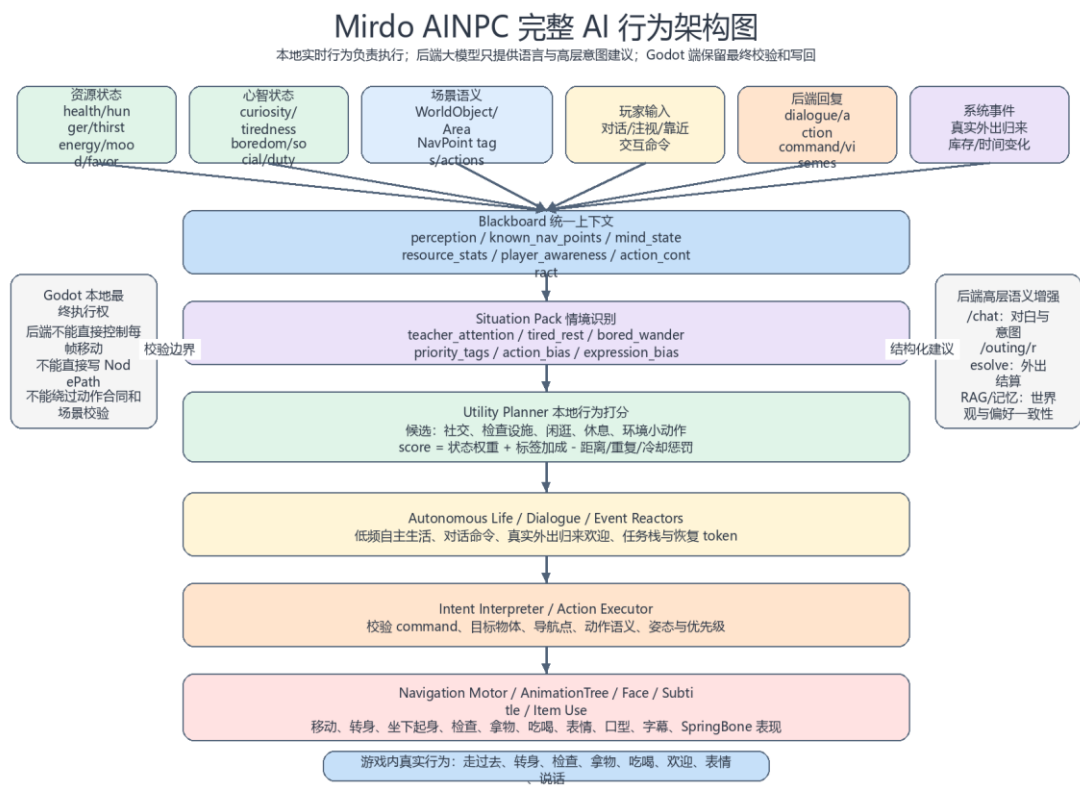


图 5.8 Mirdo AINPC 完整 AI 行为架构图

图 5.8 所示的架构图按运行时数据流绘制，而不是按源码文件夹机械罗列。最上层的资源状态、心智状态、场景语义、玩家输入、后端回复和系统事件会不断进入 Blackboard；中间的 Situation Pack 与 Utility Planner 决定 Mirdo 当前更像是该整理补给、回应老师、闲逛还是休息；下层的 Autonomous Life、Dialogue 和 Event Reactors 解决“谁触发了这次行为”；最后的 Interpreter、Executor、NavigationMotor、AnimationTree、Face、Subtitle 与 Item Use 把意图变成实际动作。这样画图的原因是为了突出本项目的边界：大语言模型不直接接管角色，真正移动、坐下、检查、拿物、表情和字幕都由 Godot 端的组件逐层校验后执行。

Mirdo AI 行为系统不同于外出 AI 结算。外出 AI 负责一次外出行动的叙事和收益，Mirdo AI 行为系统负责避难所内持续存在的角色行为：她需要看见玩家、理解柜子和导航点、判断自己是否无聊或疲劳、响应玩家自然语言命令，并把动作落实到移动、转身、检查、表情、口型和字幕上。开发过程中如果只让后端返回一段聊天文本，Mirdo 看起来仍然像一个站在场景里的问答框；如果把移动、表情、库存和对话都塞进同一个脚本，又很快会出现互相抢控制的问题。因此当前版本把智能拆成“本地可控行为”和“语

言模型高层意图”两部分：日常移动、检查柜子、注视玩家、坐下休息、补给使用等由 Godot 本地组件决定；自然语言对话、情绪和命令解释才交给后端模型辅助。

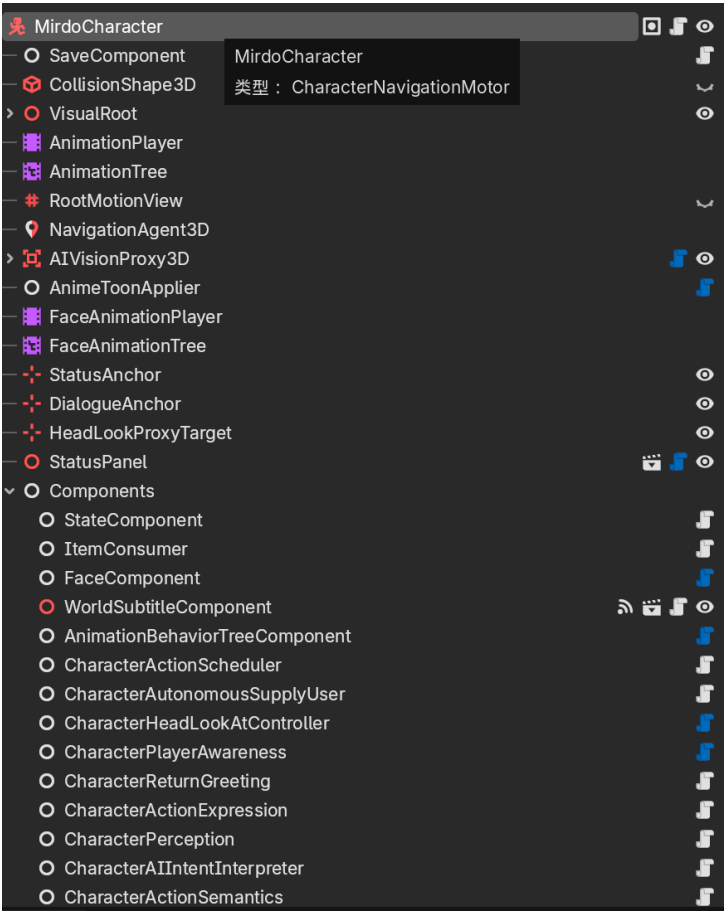


图 5.9 Mirdo 角色组合式节点结构图

从完整架构看，Mirdo 的输入并不只有玩家文本，而是由资源状态、心智状态、场景语义、玩家输入、后端回复和系统事件共同组成。系统先由 Blackboard 统一上下文，再由 Situation Pack 识别情境，由 Utility Planner 进行本地打分，由 Autonomous Life、Dialogue 和 Event Reactors 处理日常行为、对话命令和事件反应，最后经过 Intent Interpreter、Action Executor、Navigation Motor、AnimationTree、Face、Subtitle 和 Item Use 落到游戏内真实行为。也就是说，玩家看到的“走过去、转身、检查、拿物、吃喝、欢迎、表情和说话”并不是单个脚本随机播放动画，而是多层组件连续协作后的结果。

Mirdo AI 可以按多层架构理解。第一层是输入感知层，由 CharacterPerceptionComponent、AIVisionProxy3D、AIWorldObjectComponent、AIPerceptionArea3D 和 CharacterAINavPoint3D 构成，负责把“玩家在哪里、场景里有什么、哪些点可以到达、对象有什么语义标签”转换成可读数据。第二层是 AI 黑板层，由 CharacterAIBlackboardComponent 汇总 actor、perception、known_nav_points、

mind_state、resource_stats、player_awareness、current_behavior、action_contract 和 situation_context，形成统一 runtime snapshot，既供本地规划使用，也供 /chat 请求作为上下文。第三层是动作语义层，由 CharacterActionSemanticsComponent 维护 idle、locomotion、turn、posture、work、reaction、ambient、social 等动作的 category、posture、loop、interruptible、uses_root_motion、default_expression、tags 和 description，避免模型或规划器选择不存在或姿态不匹配的动作。

第四层是情境与心智层。CharacterMindStateComponent 表示好奇、疲劳、无聊、社交、责任感和警惕等内部变量；CharacterSituationBehaviorPackComponent 根据玩家注视、距离、精力、疲劳、无聊和好奇生成 teacher_attention、tired_rest、bored_wander 等情境包，并输出 priority_tags、avoid_tags、action_bias、expression_bias 和 decision_bias。第五层是规划决策层，由 CharacterAutonomousPlannerComponent 根据黑板、心智状态、情境包、对象标签、距离、重复惩罚和冷却时间产生候选行为。第六层是自主生活层，由 CharacterAutonomousLifeComponent 以低频思考方式处理任务栈、外部命令宽限、恢复 token、移动冷却、到达停留、自言自语和恢复任务，避免玩家命令被自主行为立即覆盖。

第七层是任务调度和动作执行层。CharacterActionSchedulerComponent 负责按优先级排队动作和返回动作，CharacterAllIntentInterpreterComponent 负责把后端返回或本地决策转换为统一意图，CharacterAIActionExecutorComponent 负责寻找目标物体、目标 marker、导航点、跟随目标和动作参数。第八层是导航运动层，由 CharacterNavigationMotor 和 NavigationAgent3D 处理 move_to_position、move_to_marker、NavMesh 投影、起步转身、walk/run 切换、到达停止、自动开门、NavMesh 外恢复以及精准坐下点。最新实现还加强了坐下起身后的导航恢复：stand_resume_navigation_delay_sec、stand_ready_max_planar_distance、suppress_next_navigation_turn_state 和独立转身逻辑用于避免角色刚起身就错误转向或卡在座位附近。第九层是表现与反馈层，包括 CharacterAnimationBehaviorTreeComponent、BlendShapeFaceComponent、CharacterActionExpressionComponent、CharacterHeadLookAtController、CharacterPlayerAwarenessComponent、CharacterReturnGreetingComponent、WorldSubtitleComponent 和 CharacterAutonomousSupplyUserComponent，负责动画状态、表情口型、动作表情联动、头部注视、玩家注视反应、外出归来欢迎、世界字幕和自主补给使用。

从架构设计看，MirDo 不是一个集中式超大脚本，而是组合式节点体系。角色根节点 MirDoCharacter 负责承载 CharacterBody3D、模型、导航、动画和组件容器；每个组

件只处理一个相对独立的职责，并通过 **NodePath**、信号和运行时查询协作。感知组件不直接播放动画，规划组件不直接改库存，后端对话组件不直接移动角色，执行组件也不直接生成语言。这样做的好处是：第一，角色行为可以逐步扩展，例如新增“整理工作台”只需要新增语义点和动作语义，而不必改动所有 AI 逻辑；第二，AI 生成内容被限制在意图层，动作是否合法仍由本地组件判断；第三，动画、口型、SpringBone 等表现层可以独立调试，不会破坏规划逻辑。

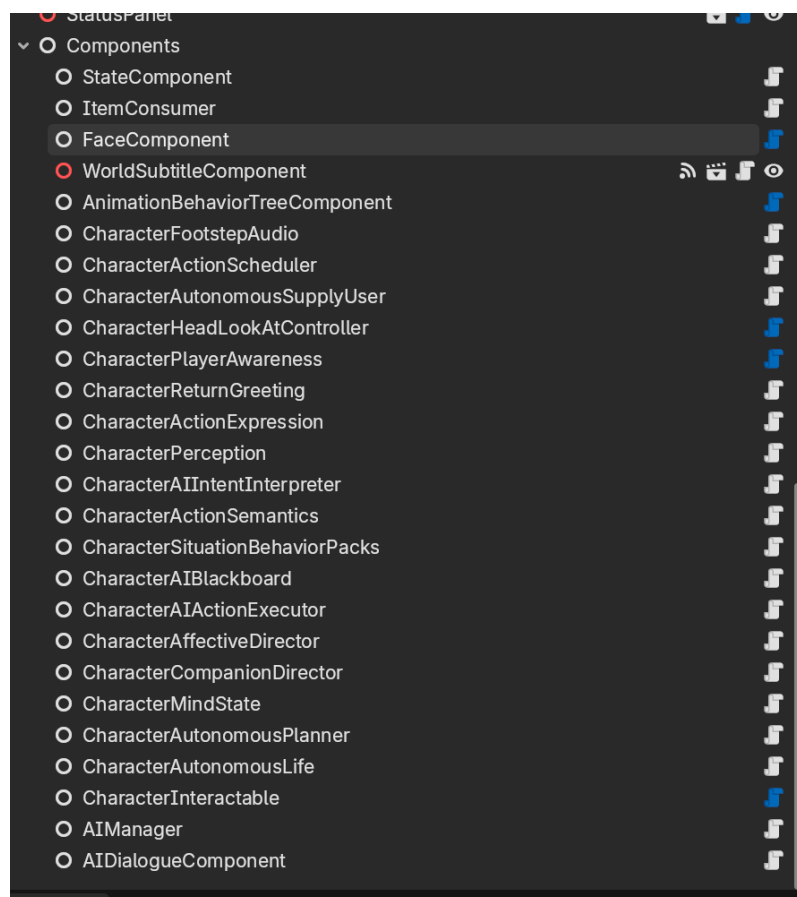


图 5.10 Mirdo 角色 AI 组件节点结构补充图

完整架构中还有一个关键点是“场景物体也参与 AI”。食品柜、医疗柜、工具箱、工作台、床边、门口观察点和玩家附近点并不是普通装饰物，而是通过 **AIWorldObjectComponent**、**AIPerceptionArea3D** 和 **CharacterAINavPoint3D** 向 Mirdo 暴露 **object_id**、**display_name**、**ai_description**、**object_type**、**tags**、**supported_actions**、**marker_roles**、**arrival_action**、**arrival_expression**、**cooldown_sec** 和 **dwell_time_sec** 等语义。这样 Mirdo 不需要硬编码某个节点路径，而是根据 **food**、**medical**、**tool**、**storage**、**seat**、**rest**、**social** 等标签理解“这个地方能满足什么需求、到达后该做什么动作”。这种设计接近 **Smart Object / Smart Terrain** 的思路：世界对象主动提供可执行语义，AI 只需

要读取语义并进行选择。

上述分层的设计依据可以对应到几类已有 AI 方法。第一，黑板架构对应 **CharacterAIBlackboardComponent**：它不直接做决策，而是把感知、心智、资源、玩家注视、当前行为、情境包和动作合同放在同一个 runtime snapshot 中，供本地规划和后端 prompt 共同读取^{[8][22]}。第二，效用 AI 对应 **CharacterAutonomousPlannerComponent**：MirDo 不通过大量 if-else 写死行为，而是让 food、medical、tool、seat、wander、social 等标签与 curiosity、tiredness、boredom、social、duty、caution 等心智变量共同参与评分^[18]。第三，行为树思想对应动作语义和动画状态组织：动作不是模型随意生成的字符串，而是被拆成 posture、work、react、locomotion、idle 等层级，再由调度器和执行器判断当前姿态是否允许播放^{[12][20]}。第四，实时规划思想对应后端 **CharacterBehaviorPlanner** 的目标修复逻辑：玩家说“去看看食物还够不够”时，后端先识别目标，再把它规范到 food_cabinet 或对应 nav point，最后只返回客户端可执行的 command 和 payload^[19]。第五，大模型 Agent 思想对应 **AIDialogueComponent** 与 **CharacterBehaviorPlanner**：模型负责把玩家自然语言转为可执行意图，后端再做动作、表情、口型和目标修复，最终仍由 Godot 端执行与拒绝非法结果^{[13][14]}。第六，RAG 与长期记忆用于保持 MirDo 的人格、世界观、外出规则和玩家偏好一致，不用于直接绕过客户端规则^[15]。第七，组合式节点对应 **Component Pattern**：MirDo 的 Mind、Perception、Planner、Life、Executor、Navigation、Face、Dialogue 等能力被拆为独立 Godot 节点，符合用组件拆分游戏对象职责、降低巨型脚本复杂度的思想^[23]。

MirDo 的一次自主行为通常经过“感知快照—黑板汇总—情境包评估—候选生成—评分选择—任务调度—动作执行—表现反馈—状态回写”九个步骤。这里的流程不是论文中临时抽象出来的，而是实际调试中逐步拆开的：**CharacterPerceptionComponent** 先收集 **AIWorldObjectComponent**、**AIPerceptionArea3D** 和 **CharacterAINavPoint3D**，形成带标签、距离、可执行动作和到达动作的候选环境；**CharacterAIBlackboardComponent** 再把这些对象与玩家距离、资源状态、当前行为、动作合同和情境包组合成上下文；**CharacterSituationBehaviorPackComponent** 根据玩家注意力、疲劳和无聊等因素给出偏置；**CharacterAutonomousPlannerComponent** 根据 curiosity、tiredness、boredom、social、duty、caution 等变量生成候选行为；最后由 **AutonomousLife** 判断外部命令宽限、恢复 token、任务栈和冷却时间，交给 **ActionExecutor** 与 **NavigationMotor** 执行导航、坐下、起身、看向、播放动作和设置表情。

Planner 的精髓在于“语义标签 + 心智权重 + 冷却约束”的 Utility AI 打分。场景中的食品柜、医疗柜、工具箱、座椅、床边、门口观察点等不是写死在 if-else 中，而是通过 food、supplies、medical、tool、material、seat、rest、wander、social 等标签进

入候选集。责任感 `duty` 高时，带 `supplies`、`medical`、`storage` 标签的目标会获得更高分；疲劳 `tiredness` 或能量偏低时，`seat`、`rest`、`bed` 类目标更容易被选择；无聊 `boredom` 和好奇 `curiosity` 高时，`wander`、`corner`、`tool`、`material` 等目标更容易触发巡看或检查；社交 `social` 高或玩家持续注视时，看向玩家、挥手、点头、歪头等社交动作会得到额外分。与此同时，`distance penalty`、`repeat target penalty`、`repeat kind penalty`、`target cooldown` 和 `same target cooldown` 会压低距离过远、刚刚执行过或过于重复的目标。调试时最明显的效果是：MirDo 不再总是站桩，也不会像随机动画一样连续做同一个动作，而是在“想整理补给、想靠近玩家、想休息、想看看房间”之间形成可解释的变化，这种按情境和需要动态加权的做法与效用 AI 的评分选择思想一致^[18]。

动作语义表使“能做什么”和“应该怎样做”有了统一描述。例如 `sit_down`、`stand_up` 标记为 `posture` 和 `root_motion`，`work_count_supplies` 标记为 `supplies`、`food`，`rub_eye` 和 `sleepy_yawn` 标记为 `tired`，`tiny_wave`、`small_wave`、`small_nod`、`tilt_head_cute` 标记为 `teacher` 或 `cute`。后端 `PromptBuilder` 会把 `action_contract` 传给模型，`CharacterBehaviorPlanner` 也会把模型输出规范到可用动作、表情和口型列表中。这样“去看看食物柜”最终不会被解释成去椅子休息，而会倾向 `go_to_object` 或 `go_to_nav_point`，并选择 `work_count_supplies`。

`AutonomousLife` 是让 MirDo “像在生活”的关键层。它并不每帧请求决策，而是以 `think_interval_min`、`think_interval_max` 控制低频思考；当玩家刚对话、下命令或触发交互后，`external_grace_sec` 会暂时阻止自主行为抢占；`movement_cooldown_sec`、`same_target_cooldown_sec` 和 `sit_cooldown_sec` 用来避免短时间频繁移动、重复访问同一目标或反复坐下起身；`task_stack` 与 `resume token` 用于保存被玩家对话、后端命令或欢迎事件软打断的旧任务，等宽限期结束后再尝试恢复。到达导航点后，组件还会根据 `dwel_time_sec` 停留，并以较低概率触发 `self_talk` 自言自语。这样 MirDo 在避难所内会呈现“偶尔观察、偶尔整理、偶尔靠近玩家、累了休息、被打断后能继续”的节奏，而不是高频抖动或完全静止。

MirDo 场景 `mirDo_character.tscn` 中挂载了完整的 AI 行为链。`CharacterMindStateComponent` 维护好奇、疲劳、无聊、社交、责任感和警惕等内部状态；`CharacterPerceptionComponent` 读取场景中的 AI 语义对象、感知区域和导航点；`CharacterAIBlackboardComponent` 将角色位置、可见对象、导航点、资源状态、玩家距离、当前行为和动作契约组织成统一上下文；`CharacterAutonomousPlannerComponent` 根据内部状态、对象标签、距离、重复惩罚和冷却时间对候选行为打分；`CharacterAutonomousLifeComponent` 周期性触发自主思考，并通过 `external_grace_sec`、`movement_cooldown_sec`、`same_target_cooldown_sec` 等参数避免玩家刚下命令后被自主

行为打断。

场景语义由 `AIWorldObjectComponent`、`AIPerceptionArea3D` 和 `CharacterAINavPoint3D` 提供。例如，食品柜导航点可以带有 `food`、`storage`、`supplies`、`cabinet` 标签，到达后播放 `work_count_supplies` 并显示较活泼的表情；医疗柜导航点到达后播放 `work_check_shelf`；装备柜或工具箱导航点到达后播放 `work_inspect_cabinet` 或 `work_check_lower`；工作台导航点到达后播放 `work_explain`；床边或椅子点用于 `tired_rest` 情境下的坐下、揉眼或打哈欠；门口观察点用于 `look_around`；玩家附近点用于 `tiny_wave`、`small_nod` 或靠近玩家。这些语义标注让 `Mirdo` 能够理解“食品柜”“医疗柜”“工具箱”等对象的用途，也让新增设施时只需要补充语义组件和导航点，而不是重写 `Planner`。

`Mirdo` 行为调试时主要关注两个现象：一是角色不能频繁来回移动，二是不能在玩家刚发出命令后立刻被自主行为覆盖。为此 `AutonomousLife` 中保留 `external_grace_sec`、`movement_cooldown_sec`、`same_target_cooldown_sec`、`sit_cooldown_sec` 和 `task_stack` 等参数。调试时如果发现 `Mirdo` 反复去同一个柜子，就提高同目标冷却或重复惩罚；如果角色太安静，则提高 `boredom`、`curiosity` 对 `ambient` 和 `wander` 的权重；如果对话命令被打断，则延长外部命令宽限。论文中把这些参数写出，是为了说明行为系统不是一次性随机动画，而是经过约束的低频自主行为。

玩家与 `Mirdo` 对话时，`CharacterAIDialogueComponent` 会构造包含 `NPC` 合同、`Blackboard`、`perception`、`known_nav_points`、`action_contract`、可用动作、可用表情和可用口型的请求。后端根据上下文返回结构化 `JSON`，包括 `dialogue`、`emotion`、`expression`、`action`、`command`、`command_payload`、`visemes` 或 `viseme_sequence` 等字段。`CharacterActionSemanticsComponent` 定义动作语义和动作适用场景，`CharacterAIIntentInterpreterComponent` 将模型返回解释为统一意图，`CharacterAIActionExecutorComponent` 再负责执行导航、跟随、看向玩家、坐下、起身、播放动作和设置表情。该链路保证模型只能从动作契约中选择能力，不能凭空要求客户端执行不存在的动作，也不能直接返回 `Godot NodePath` 来越权控制场景。

`Mirdo` 的身体动作由 `CharacterAnimationBehaviorTreeComponent` 控制。该组件将 `idle_normal`、`idle_relaxed`、`idle_sleepy`、`listen`、`walk`、`run`、`seated_idle`、`work_inspect_cabinet`、`work_check_shelf`、`work_take_item`、`work_drink`、`react_nod`、`tiny_wave`、`tilt_head_cute`、`look_around`、`turn_left`、`turn_right` 等动作映射到 `AnimationTree` 状态机。`CharacterActionSchedulerComponent` 在角色忙碌时按优先级排队动作，并支持 `return_action`，可用于“先检查、再拿物、再喝水”这样的短动作链。角色移动不是直接改位置，而是由 `CharacterNavigationMotor`、`NavigationAgent3D` 与 `RootMotion` 配合：起

步前判断是否需要转身，移动中根据速度切换 walk/run，到达后停止动作，必要时自动开门，从 NavMesh 外恢复到 NavMesh；坐下和起身还处理座位标记对齐、root motion 结束等待、起身后重新导航等细节。表情和口型由 BlendShapeFaceComponent 控制，支持 neutral、joy、fun、angry、sorrow、surprised 和 aa、ih、ou、E、oh 五类口型。

此外，CharacterAutonomousSupplyUserComponent 能在 Mirdo 饥饿或口渴较低时尝试寻找食物或水。它会判断当前需要、寻找带有 food、supplies、storage、cabinet 标签的补给对象，导航到目标，播放检查、拿取、饮用等动作，并将物品效果应用到 Mirdo 状态。CharacterActionExpressionComponent 负责动作与表情联动，例如 small_wave 倾向 face_joy，work_count_supplies 倾向 face_fun，rub_eye 或 sleepy_yawn 倾向困倦表情，同时尊重后端显式 expression，避免刚设置的表情立刻被动作覆盖。

CharacterPlayerAwarenessComponent 会检测玩家距离和视线注视，当玩家靠近、非常靠近或持续注视 Mirdo 时触发 tiny_wave、small_nod、tilt_head_cute、cute_startle 等反应，并请求头部 LookAt 与表情变化。CharacterHeadLookAtController 通过代理目标和 SkeletonModifier3D 控制头部朝向，根据待机、对话、移动、工作、坐姿和反应等上下文调整权重，使角色不会在工作动作中僵硬盯着玩家，也不会对话时视线游离。

WorldSubtitleComponent 负责世界空间字幕、打字机效果、自动分页和说话时口型联动，使后端对白、本地自言自语和欢迎事件都能以角色身体旁边的字幕呈现。该模块体现了角色 AI 与库存系统、状态系统、玩家感知和表现系统的联动。

对话行为与自主行为共享同一套执行边界。AIDialogueComponent 向后端发送的不是任意场景树，而是经过整理的 NPC 合同和黑板上下文：包含可用动作、表情、口型、known_nav_points、perception、mind_state、resource_stats、player_awareness、current_behavior 和 action_contract。后端返回 command 与 command_payload 后，CharacterAllIntentInterpreterComponent 会把 follow_player、stop_follow、look_at_player、go_to_nav_point、go_to_object、sit_down、stand_up、play_action、set_expression 等命令转成统一意图；CharacterAIActionExecutorComponent 再根据场景中真实存在的 nav point、object id、marker role 和动作语义执行。如果模型返回不存在的对象、错误动作或越界命令，执行层会忽略或降级为本地提示。因此 Mirdo 的“大语言模型智能”不是绕过 Godot，而是被放在组合式节点架构中的高层建议位置。

在玩家体验上，Mirdo 的行为层次可以分成五类。第一类是生存协同行为，例如低状态时寻找补给、到食品柜检查余量、到医疗柜查看药品、在工作台前说明情况；第二类是环境生活行为，例如在床边打哈欠、到门口观察、在工具箱前检查、在房间角落短暂停留；第三类是社交陪伴行为，例如靠近玩家、看向玩家、挥手、点头、歪头、受到注视时害羞或惊讶；第四类是对话意图行为，例如玩家用自然语言要求她跟随、停止跟

随、去检查某个柜子或做出某个动作；第五类是真实外出归来反馈，玩家从外出结算返回避难所后，CharacterReturnGreetingComponent 会读取 Global 记录的真实外出事件，靠近玩家或原地欢迎，播放 tiny_wave、small_wave、small_happy_bounce 等轻量动作，并提高心情和好感。实际实现中这五类行为不是五套互相孤立的脚本，而是共用 Blackboard、ActionScheduler、ActionExecutor、NavigationMotor 和表现层，所以同一个“去食品柜”目标既可以来自自主生活，也可以来自玩家命令，还可以被外出归来事件暂时打断后恢复。

从调试和可观测性看，完整架构也保留了明确的日志入口。CharacterAIBlackboardComponent 提供 print_debug_snapshot() 和 print_debug_snapshot(true)，前者输出当前主要情境、行为、目标、任务栈、精力、心情、注视和距离状态，后者输出完整 LLM context JSON。运行时还可以通过 [AIBlackboard]、[GlobalOuting]、[MirdoReturnGreeting]、[CharacterNavigationMotor]、[CharacterAutonomousPlanner] 和 [CharacterAutonomousLife] 等日志定位问题。例如，如果 Mirdo 总是去同一个柜子，可以从 Blackboard 与 Planner 日志查看标签权重、冷却和重复惩罚；如果玩家命令被覆盖，可以查看 external_grace_sec、task_stack 和当前行为锁。论文中保留这些调试点，是为了说明 AI 行为系统不仅能演示，也能解释和排查。

5.9 Mirdo 角色表现与 SpringBone 物理模拟实现

Mirdo 的智能性不仅体现在决策层，也体现在表现层。若角色只会返回文本，即使 AI 回复内容正确，也容易给玩家一种“菜单式 NPC”的感觉。因此项目在角色场景中同时实现了身体动画、表情口型、头部注视、二级物理和卡通渲染。Mirdo 场景中包含 AnimationPlayer、AnimationTree、RootMotionView、NavigationAgent3D、FaceAnimationPlayer、FaceAnimationTree、HeadLookProxyTarget 和 Components 节点，AI 行为最终会落到这些表现节点上。

角色身体动作使用 AnimationTree 统一管理。移动类动作通过 walk、run、stand_to_walk、walk_to_run、run_to_stop 等动画表达速度变化；工作类动作包括 inspect_cabinet、check_shelf、check_lower、count_supplies、take_item、place_item、drink、cute_explain；社交类动作包括 small_wave、tiny_wave、small_nod、tilt_head_cute、cute_startle、look_back 和 look_around。动作执行层不会直接播放任意动画，而是经过动作语义表和调度器判断，保证坐姿动作、站姿动作、移动动作和工作动作不冲突。

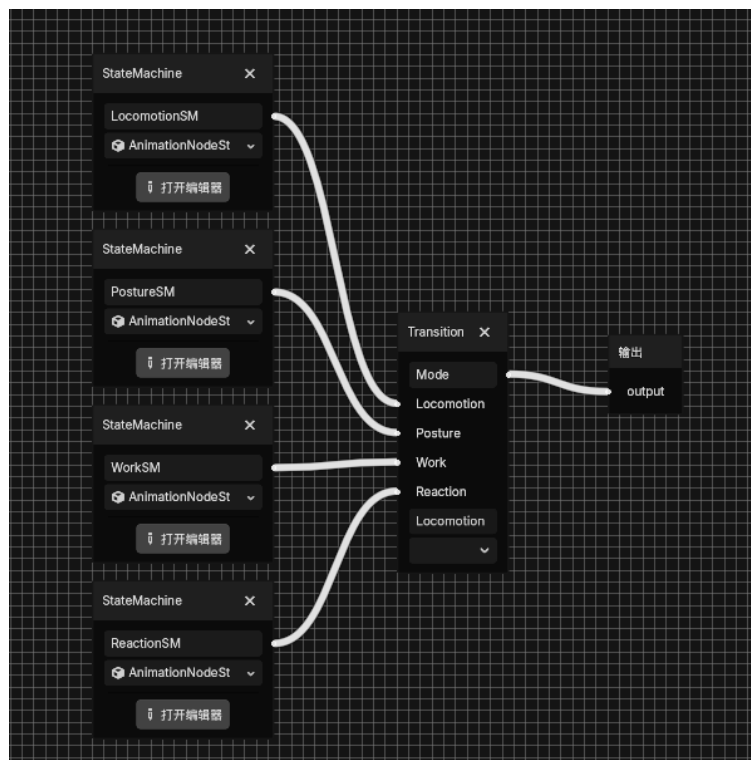


图 5.11 Mirdo 动画行为树根节点

表情口型系统基于 BlendShapeFaceComponent。该组件将 face_neutral、face_joy、face_fun、face_angry、face_sorrow、face_surprised 等抽象表情映射到模型 BlendShape，并通过 FaceAnimationTree 控制 ExpressionSM、VisemeSM、TalkBlend 和 BlinkBlend。对话接口返回的 visemes 会被转换为 aa、ih、ou、E、oh 等口型序列，自动眨眼由 blink_interval_min、blink_interval_max、blink_duration 等参数控制。这样 Mirdo 说话时不仅有字幕和语音式口型节奏，也有符合情绪的表情过渡。

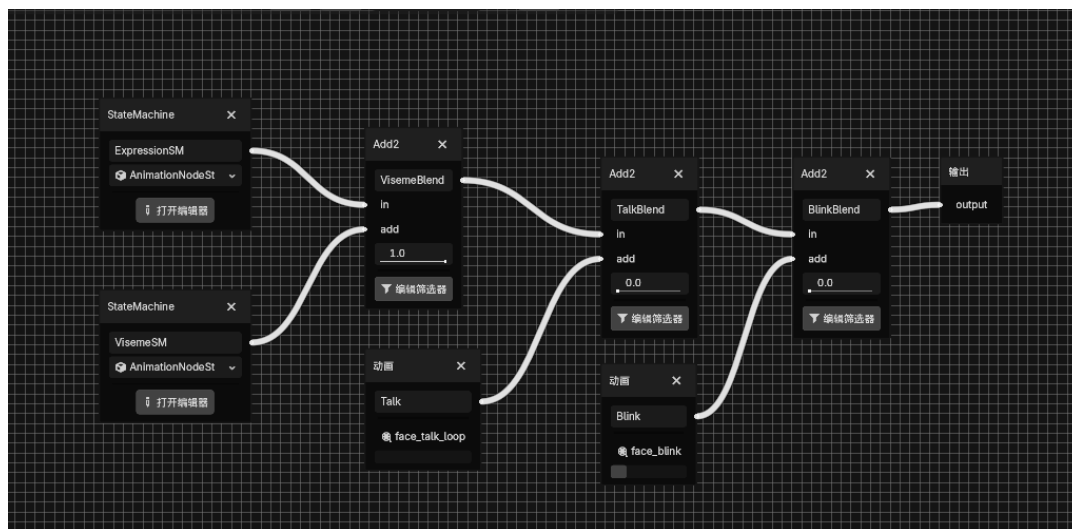


图 5.12 面部表情与口型动画树

SpringBone 物理模拟是角色表现的重要补充。Mirdo 场景在 GeneralSkeleton 下挂载 SpringBoneSimulator3D 节点 AnimeSpringBones, influence 设置为 0.78, 包含 18 条模拟链。模拟对象覆盖呆毛、后发、前刘海、侧发、发束、发饰以及外套/裙摆下摆等部位, 例如 hair_aho 到 hair_aho.001、hair_back.001.L 到 hair_back.002.L、hair_front 到 hair_front.001、A_hem.001 到 A_hem.003、B_hem.001.L 到 B_hem.003.L 等。每条链配置 radius、stiffness、drag、gravity 和 gravity_direction, 用来控制摆动半径、回弹刚度、阻尼和重力下垂。头发链阻尼较低、重力较小, 以表现轻微摆动; 衣摆链刚度和阻尼更高, 重力更明显, 以避免移动时穿模或过度飘动^[6]。

为了减少二级运动穿模, 场景中还在 AnimeSpringBones 下配置了 SpringBoneCollisionSphere3D 与 SpringBoneCollisionCapsule3D 碰撞体, 包括 HeadSphere、FaceSphere、NeckSphere、ChestCapsule、WaistCapsule、HipsCapsule、左右上臂、左右下臂、左右大腿和小腿等。碰撞体绑定到 Head、Neck、UpperChest、Spine、Hips、LeftUpperArm、RightUpperArm、LeftUpperLeg、RightUpperLeg 等骨骼, 并设置 radius、height 和 position_offset。SpringBone 链启用 enable_all_child_collisions 后, 会使用这些碰撞约束, 使头发、衣摆在角色移动、转身、坐下和起身时绕开身体主要部位。

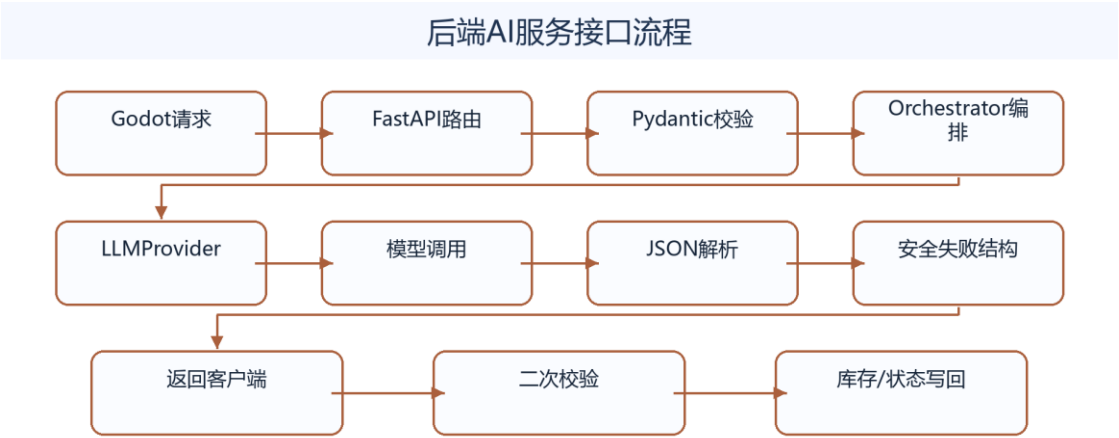
项目还保留了 AnimeSpringBoneApplier 与 AnimeSpringBoneProfile 资源化配置方式。res://features/character_physics/mirdo_springbone_profile.tres 记录 SpringBone 链、碰撞体、影响权重和骨骼路径; res://features/character_physics/anime_springbone_applier.gd 可根据 profile 自动创建 SpringBoneSimulator3D、碰撞体节点和链参数。当模型骨架从 Skeleton3D 重命名为 GeneralSkeleton 时, 脚本会通过 fallback_skeleton_paths 和递归查找进行兜底, 避免导入资源变化导致物理失效。SpringBoneCollisionMirror3D 还可以把 SpringBoneCollision3D 镜像为 StaticBody3D 或 AnimatableBody3D 的 CollisionShape3D, 用于调试或与 SoftBody3D/代理布料碰撞配合^[6]。

SpringBone 与 AI 行为之间存在直接的表现衔接。Mirdo 走向柜子时, NavigationAgent3D 和 RootMotion 决定身体位移, SpringBone 让头发、发饰和衣摆在加速、减速、转身时产生滞后摆动; Mirdo 坐下和起身时, 主动画负责骨骼姿态, SpringBone 负责衣摆与头发的惯性回弹; Mirdo 受到玩家注视后歪头或挥手时, HeadLookAt 与 BlendShapeFaceComponent 改变视线和表情, SpringBone 则提供轻微二级运动, 使“害羞、惊讶、回应”这些 AI 反馈更可信。换言之, 物理模拟服务于智能 NPC 的可感知反馈, 而不是孤立的美术装饰。

从论文实现角度看, SpringBone 并不是单纯美术效果, 而是 Mirdo 智能 NPC 可信表现的一部分。AI 行为产生移动、转身、挥手、检查柜子、坐下和起身等动作;

AnimationTree 负责主骨骼动画；BlendShapeFaceComponent 负责表情与口型；HeadLookAtController 负责视线；SpringBoneSimulator3D 负责头发和衣摆的二级运动。多层表现叠加后，玩家看到的不是只会输出文字的 NPC，而是一个会看向玩家、会做动作、会说话、会因运动产生自然摆动的角色。

5.10 后端 AI 服务设计与实现



说明：图中箭头表示数据或控制流程的主要方向。

图 5.13 后端 AI 服务接口流程图

后端服务放在 `server://` 目录下，使用 FastAPI 实现[7]。它不是多人联机服务器，而是给 Godot 提供本地 AI 能力：调用模型、拼接提示词、解析 JSON、保存会话记忆、检索知识库、整理角色行为以及处理外出结算。真正改变游戏状态的仍然是客户端。`app/main.py` 中注册了 `/health`、`GET /model/probe`、`/chat`、`/outing/resolve`、`/session/{session_id}/history`、`/session/{session_id}/snapshot`、`/memory/clear` 和 `/ingest` 等接口，请求和返回值通过 Pydantic 模型约束，避免 Godot 端收到随意变化的字段[16]。

服务启动时通过 `lifespan` 初始化运行目录、SQLite 记忆库、RAG 检索器、`MemoryRAGRetriever`、`LLMProvider`、`ChatOrchestrator` 和 `ExpeditionOrchestrator`。`Settings` 负责读取服务名、版本、运行目录、知识库目录、模型默认配置、监听地址和端口；`MemoryStore` 初始化本地对话数据库；`RAGRetriever` 连接世界知识 Chroma 索引；`MemoryRAGRetriever` 把同一 `session` 的长期记忆以语义方式检索出来；`LLMProvider` 统一解析模型提供方配置。这样的启动流程使后端可以独立于 Godot 运

行，也可以配合 Godot 的 AISettingsPanel 做健康检查和模型探测。

/health 是客户端最先访问的接口，用于返回 ok、service、version、llm_ready、rag_ready、memory_ready、runtime_dir、knowledge_dir、host 和 port 等状态。Godot 可以根据该接口判断后端是否可用。/model/probe 采用 GET 请求，用于探测后端当前模型配置是否可用。AISettings.test_provider_connection 会先访问 /health，再 GET /model/probe，并记录服务延迟和模型延迟。这样玩家在主菜单或设置面板中就能确认本地后端和远端模型是否连通。



图 5.14 Mirdo 对话与记忆测试界面

LLMProvider 是后端连接不同大模型服务的适配层。Godot 可以在请求中携带 provider，也可以使用用户设置中的 base_url、api_key、model 和 proxy_url。后端会清理 base_url 的结尾斜杠，判断 provider 是否完整，并在调用模型时设置 max_tokens、timeout 和代理。这样项目并不绑定某一个固定模型服务，而是兼容 OpenAI-compatible 接口。由于大模型输出存在不确定性，后端不会把模型文本直接交给游戏执行，而是要求模型按 JSON schema 输出，并在服务端进行解析和兜底^[9]。

ChatOrchestrator 负责 Mirdo 对话流程。一次 /chat 请求中，Godot 会发送 session_id、玩家输入、时间、NPC 状态、给出的物品、感知上下文、黑板上下文、可用动作、动作语义合同、可用表情和可用口型。后端先把玩家回合写入 MemoryStore，

再读取最近对话、长期记忆和知识库检索结果，构造包含角色设定、行为边界、动作契约、真实外出归来事件和输出格式要求的 `prompt`。模型返回后，`response_parser` 将文本解析为结构化字段，如 `dialogue`、`emotion`、`expression`、`action`、`command`、`command_payload`、`visemes` 和 `memory_updates`。若模型输出缺字段或格式异常，后端会进入 `CharacterBehaviorPlanner` 或本地 `fallback`，`Godot` 端也会触发本地 `fallback` 文案。

后端没有被设计成大型服务器，也没有保存玩家的权威游戏状态。它更像随游戏运行的本地 `companion service`：/`health` 用于让客户端知道服务是否可用，/`model/probe` 用于提前发现 `base_url`、`model` 或 `api_key` 配置问题，/`chat` 面向 `MirDo` 对话，/`outing/resolve` 面向外出结算，/`session` 和 /`memory` 接口用于调试记忆。这样的设计方便毕业设计演示：即使模型服务暂时不可用，`Godot` 仍能通过本地保守结算和失败提示保持主循环可运行。

对于真实外出归来事件，后端也做了边界限制。如果 `ChatRequest.context` 中包含 `event=real_outing_return` 且 `real_outing=true`，`PromptBuilder` 会把 `location_name`、`total_minutes`、`loot_added`、`health_damage` 和 `risk` 等字段写入 `runtime state`；`CharacterBehaviorPlanner` 会清空移动类 `command`，禁止 `go_to_object`、`go_to_nav_point` 和 `follow_player`，只允许 `tiny_wave`、`react_wave`、`happy_bounce`、`listen` 等轻量欢迎动作。真正的靠近玩家、看向玩家和欢迎台词显示由 `Godot` 的 `CharacterReturnGreetingComponent` 完成，后端只负责短对白、表情和轻量动作建议。

`ChatOrchestrator` 的重点不是让模型直接控制 `MirDo`，而是让模型在上下文内选择“可执行意图”。例如玩家说“去看看食物还够不够”，模型可以返回 `command` 为前往食品柜、`action` 为 `work_count_supplies`、`expression` 为 `joy` 或 `neutral`，但真正的导航目标是否合法、动作是否存在、角色是否需要先起身，都由 `Godot` 的 `CharacterAllIntentInterpreterComponent`、`CharacterAIActionExecutorComponent` 和动作语义表处理。这种设计对应 `ReAct` 中“推理与行动结合”的思想，但在游戏中进一步加入动作白名单和客户端校验，避免语言模型越权控制角色^[14]。

`CharacterBehaviorPlanner` 是后端新加入的确定性行为后处理层。它会把 `action` 规范到 `MirDo` 的动作集合，把 `expression` 规范到 `neutral`、`joy`、`fun`、`angry`、`sorrow`、`surprised` 等可用表情，把 `visemes` 限制到 `aa`、`ih`、`ou`、`E`、`oh` 五类口型；同时根据玩家文本、`perception` 和 `known_nav_points` 修复 `go_to_object` / `go_to_nav_point` 的 `payload`。例如玩家说“去看看食物柜”时，如果当前感知里有 `food_cabinet`，就优先返回 `target_object=food_cabinet`、`marker_role=approach` 和 `work_count_supplies`；如果当前感知没有但 `known_nav_points` 有食物柜导航点，则改用 `go_to_nav_point`。模型调用失败

时, `behavior_planner.local_fallback_response` 也能生成跟随、停止跟随、检查设施等基础可执行意图。

`ExpeditionOrchestrator` 专门负责 `/outing/resolve`。Godot 端会发送地点规则、携带物、时间、`available_loot`、可解锁邻居和 `provider` 配置。后端构造紧凑 `prompt`, 要求模型只输出 JSON, 字段包括 `ok`、`title`、`summary`、`story`、`experience`、`risk_result`、`loot`、`discovered_clues`、`mood`、`health_damage` 和 `fallback`。其中 `story` 是核心输出, 提示词要求按离开庇护所、靠近地点、搜索、遭遇丧尸或环境意外、携带物与资源取舍、撤离返程的顺序生成连续叙事, 而不是只给奖励清单。

为了保证外出收益可信, 后端会根据 `available_loot` 建立合法资源路径集合。模型返回的 `loot` 必须使用这些路径, 数量也会被限制, 非法 `item_path` 会在 `_parse_response()` 中被过滤。`health_damage` 会被限制在 0 到 35 之间, `experience` 为空时生成 `fallback` 短句, `summary` 和 `risk_result` 缺失时也会根据地点与威胁生成保守文本。若模型调用抛出异常、返回空内容、达到 `token` 限制或 JSON 无法解析, 后端返回 `ok=false`、`fallback=false` 的安全失败结构, 提示“后端 AI 未完成结算, 行动保持在出发前状态”。Godot 收到该结构后不会扣物、不会推进地图、不会保存异常结果。

^[15]后端还包含 RAG 与长期记忆能力。`/ingest` 可以将 `data/knowledge` 写入 Chroma 知识索引, `/session/{session_id}/history` 和 `/session/{session_id}/snapshot` 可以查看会话历史、最近对话和长期记忆, `/memory/clear` 可以清空指定会话或全部记忆。当前知识库包括 `worldview.md`、`mirdo_home_and_outside_contrast.md`、`outing_zombie_survival_rules.md` 和 `mirdo_action_sheet.md` 等资料, 用于强调“外面危险、家里温暖、Mirdo 等老师回来”的世界基调。记忆层从简单最近记录升级为 MemoryRAGRetriever + SQLite 相关性检索: 玩家较早说过“喜欢罐头汤”之类偏好时, 后续询问“你还记得我喜欢吃什么吗”也能被检索出来。

从安全边界看, 后端所有返回都只是“建议结果”。Mirdo 对话返回建议动作和表情, 外出结算返回建议故事、收益和伤害; 真正的库存扣除、战利品入库、状态变化、时间推进、地图解锁和存档保存全部在 Godot 端完成。后端默认监听 127.0.0.1, 适合作为本地 `companion service` 随游戏启动。Godot 端 `AIServiceSupervisor` 会先检查健康状况, 若服务不可用则尝试使用 `server://.venv/Scripts/python.exe` 或系统 `python` 启动 `run_server.py`。

后端与 Mirdo 架构的关系可以概括为“语言增强, 不替代本地 AI”。后端 `ChatOrchestrator` 负责根据角色设定、近期记忆、场景快照和动作契约生成对白与高层意图; 但 Mirdo 是否能到达某个点、动作是否适合当前姿态、是否需要停止跟随、是否要先站起来、是否能播放某个 `work` 动作, 全部由 Godot 本地组件判断。外出

ExpeditionOrchestrator 负责生成叙事化结算；但掉落是否合法、物品是否存在、携带物是否扣除、地图是否解锁和存档是否保存仍由客户端完成。这样的前后端分工让大语言模型发挥叙事、对话和意图理解优势，同时保留传统游戏系统对状态一致性、实时表现和可测试性的控制。

第 6 章 系统测试

6.1 测试环境

系统测试在 Windows 环境下进行。论文中 Godot 客户端路径统一使用 `res://` 表示，后端路径统一使用 `server://` 表示。Godot 测试使用 `headless` 方式运行脚本测试，后端测试使用 `pytest`。AI 服务默认地址为 `http://127.0.0.1:5678`。

6.2 功能测试

主要功能测试用例如下。

测试时重点不是追求大型压测，而是验证几个容易破坏体验的断点：外出地图保存后能否回到避难所，AI 返回空 `loot` 时是否有本地兜底，后端异常时是否不扣除携带物，存档槽能否列出和读取，Mirdo 的对话意图是否能被本地动作层拒绝非法目标。通过这些测试可以确认项目的关键闭环已经可演示；剩余问题主要集中在更多地点平衡、长时间行为重复度和模型响应速度上。

新增测试还覆盖了 AI 行为更新：`test_character_ai_blackboard_and_actions.gd` 验证黑板上下文和动作合同；`test_character_situation_behavior_packs.gd` 验证 `teacher_attention`、`tired_rest`、`bored_wander` 等情境包；`test_character_return_greeting.gd` 验证真实外出归来事件只在实际外出后触发；`test_character_action_executor_seat_exit.gd` 和 `test_character_stand_up_no_turn.gd` 验证坐下起身后的导航恢复和转身问题；后端 `test_character_behavior_planner.py`、`test_prompt_builder.py` 和 `test_chat_route.py` 验证 Mirdo 动作、表情、口型、设施目标和 `real_outing_return` 事件的约束。

编号	测试内容	预期结果
T1	主菜单进入游戏	能从主菜单开始新游戏或读取进度
T2	存档槽保存读取	保存后能恢复场景、库存、时间和地图进度
T3	物品堆叠	非武器按最大堆叠数

		堆叠，武器不堆叠
T4	柜子分类限制	食品柜、医疗柜、装备柜按分类接收物品
T5	Mirdo 使用物品	食物和水可作用于 Mirdo，医疗品强制玩家自用
T6	外出地图选择地点	点击地点后显示详情和准备按钮
T7	外出携带栏	可添加物品，受 12 格限制
T8	外出结算写回	带回物资写入庇护所库存，时间和地图进度更新
T9	AI 失败处理	后端模型失败时不扣物、不保存地图进展
T10	Mirdo AI 行为	Mirdo 能根据命令前往食品柜并播放检查动作

本文围绕“基于大语言模型的避难所游戏与智能 NPC 系统的设计与实现”这一题目，完成了《Mirdo》的核心原型。项目最终形成的不是一个完整商业游戏，而是一个围绕避难所生存循环搭建的可运行系统：玩家能进入避难所、整理柜子和物品、查看状态、选择外出地点、配置携带物、等待 AI 结算、触发真实外出归来反馈、带回物资并保存进度；Mirdo 也不只是静态模型，而是能在本地组件驱动下感知场景、汇总黑板上下文、应用动作语义和情境包、选择行为、执行动作、显示表情口型、迎接玩家归来，并通过后端获得对话和意图增强。其中 Mirdo 的最终形态更接近完整 AINPC 架构：状态层决定需要，语义感知层决定知道什么，Blackboard 统一事实，Situation Pack 解释情境，Utility Planner 选择行为，Autonomous Life 管理节奏与恢复，Dialogue 和后端负责语言增强，Interpreter、Executor、NavigationMotor、AnimationTree、Face、Subtitle 和 SpringBone 负责执行与表现。

本项目最重要的经验是把大语言模型放在“建议层”，而不是让它直接接管游戏。外出系统中，模型负责讲述探索过程并给出收益建议，客户端负责校验 available_loot、health_damage、时间推进和地图解锁；Mirdo 系统中，模型负责对白、情绪和高层命令，本地组件负责黑板、规划、导航、动作语义、姿态切换和 SpringBone 表现。这个边界让 AI 内容有变化，同时不会破坏 Resource、存档和状态系统。

项目仍然有明显不足：外出地点数量和收益平衡还需要更多实测，Mirdo 的长期记忆和任务目标仍比较初步，AI 响应速度受模型服务影响，角色美术和动作资源也还有继续统一的空间。后续如果继续完善，优先方向不是马上加入实时战斗，而是增加地点事件、携带物损坏规则、更多语义导航点、Mirdo 长期记忆、自动化行为测试和打包发布流程，使当前避难所循环更加稳定和丰富。

6.4 存档与持久化测试

存档测试重点验证外出地图作为独立场景时不会破坏避难所状态。
`test_save_slots_and_outing_time.gd` 用于验证外出地图保存时恢复场景应为避难所而不是外出地图；`test_outing_save_persistence.gd` 用于验证外出获得的水、能量棒等物资在保存和读取后仍然存在；地图探索次数和解锁状态也应进入全局存档。

6.5 测试结果分析

从已有测试和代码结构看，项目已经覆盖库存堆叠、外出收益保存、外出地图 UI、存档槽、后端健康检查和外出接口等关键路径。测试结果说明系统主要功能能够形成闭环。当前仍可继续完善的方向包括：增加 AI 行为自动化测试、外出 AI 返回异常格式测试、更多地点资源测试和长时间游玩下的状态平衡测试。

结 论

本文围绕“基于大语言模型的避难所游戏与智能 NPC 系统的设计与实现”这一题目，基于 Godot 4 和 GDScript 设计并实现了避难所生存互动游戏《Mirdo》。系统以第一人称避难所交互为基础，通过物品、库存、角色状态、时间推进和 Resource 存档构建生存管理框架；通过独立 2D 外出地图实现地点选择、携带物配置、地图解锁和探索结算；通过本地 FastAPI 后端和 OpenAI-compatible 模型实现 AI 外出叙事结算；通过 Mirdo AI 行为组件实现角色感知、自主规划、导航、动作、表情、口型和对话反馈。

从完成情况看，本项目的重点没有放在实时战斗上，而是放在避难所资源管理、外出风险选择和结算反馈上。武器、工具、药品等物品主要影响外出结果，而不是进入即时射击或敌人碰撞判定。AI 部分也保持了比较明确的边界：后端负责生成叙事和建议，客户端负责核对字段、过滤非法物品并写回存档；Mirdo 的本地行为组件负责日常动作，对话后端只补充语言和高层意图。这样的分工让角色有陪伴感，同时不至于让模型直接破坏游戏状态。

项目目前仍然是毕业设计阶段的原型，很多地方还有继续打磨的空间。比如，不同

外出地点的收益曲线还不够细，携带物损坏和风险减免可以拆得更明确；模型返回速度受网络和服务状态影响，后面需要更多缓存、超时和重试策略；Mirdo 的行为虽然已经拆成多个组件，但自动化测试还偏少，长时间游玩下的资源平衡也需要继续记录数据。后续如果继续开发，可以优先补充地点事件、角色行为包、原创美术资源和打包发布流程，让它从可演示原型逐步接近完整游戏。

参考文献

- [1] Godot Engine. Introduction to Godot[EB/OL]. Godot Engine Documentation[2026-05-21]. https://docs.godotengine.org/en/stable/getting_started/introduction/index.html.
- [2] Godot Engine. Nodes and scenes[EB/OL]. Godot Engine Documentation[2026-05-21]. https://docs.godotengine.org/en/stable/getting_started/step_by_step/nodes_and_scenes.html.
- [3] Godot Engine. GDScript reference[EB/OL]. Godot Engine Documentation[2026-05-21]. https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html.
- [4] Godot Engine. Resources[EB/OL]. Godot Engine Documentation[2026-05-21]. https://docs.godotengine.org/en/stable/getting_started/step_by_step/resources.html.
- [5] Godot Engine. Using AnimationTree[EB/OL]. Godot Engine Documentation[2026-05-21]. https://docs.godotengine.org/en/stable/tutorials/animation/animation_tree.html.
- [6] Godot Engine. SpringBoneSimulator3D[EB/OL]. Godot Engine Documentation[2026-05-21]. https://docs.godotengine.org/en/stable/classes/class_springbonesimulator3d.html.
- [7] FastAPI. FastAPI framework, high performance, easy to learn, fast to code, ready for production[EB/OL]. FastAPI Documentation[2026-05-21]. <https://fastapi.tiangolo.com/>.
- [8] Hayes-Roth B. A Blackboard Architecture for Control[J]. Artificial Intelligence, 1985, 26(3):251-321. [https://doi.org/10.1016/0004-3702\(85\)90063-3](https://doi.org/10.1016/0004-3702(85)90063-3).
- [9] OpenAI. Structured Outputs in the API[EB/OL]. OpenAI Platform Documentation[2026-05-21]. <https://platform.openai.com/docs/guides/structured-outputs>.
- [10] Yannakakis G N, Togelius J. Artificial Intelligence and Games[M]. Cham: Springer, 2018.
- [11] Millington I, Funge J. Artificial Intelligence for Games[M]. 2nd ed. Burlington: Morgan Kaufmann, 2009.
- [12] Iovino M, Scukins E, Styrud J, Ögren P, Smith C. A Survey of Behavior Trees in Robotics and AI[J]. Robotics and Autonomous Systems, 2022, 154:104096.

- [13] Park J S, O'Brien J C, Cai C J, Morris M R, Liang P, Bernstein M S. Generative Agents: Interactive Simulacra of Human Behavior[C]//Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. New York: ACM, 2023:1-22.
- [14] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations. 2023.
- [15] Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems 33. 2020.
- [16] Pydantic. Models and data validation[EB/OL]. Pydantic Documentation[2026-05-21]. <https://docs.pydantic.dev/latest/concepts/models/>.
- [17] Buckland M. Programming Game AI by Example[M]. Plano: Wordware Publishing, 2005.
- [18] Mark D. Behavioral Mathematics for Game AI[M]. Boston: Charles River Media, Course Technology Cengage Learning, 2009.
- [19] Orkin J. Agent Architecture Considerations for Real-Time Planning in Games[C]//Proceedings of the First Artificial Intelligence and Interactive Digital Entertainment Conference. Menlo Park: AAAI Press, 2005:105-110. <https://doi.org/10.1609/aiide.v1i1.18724>.
- [20] Colledanchise M, Ögren P. Behavior Trees in Robotics and AI: An Introduction[M]. Boca Raton: CRC Press, 2018.
- [21] Russell S, Norvig P. Artificial Intelligence: A Modern Approach[M]. 4th ed. Global ed. Harlow: Pearson Education, 2022.
- [22] Erman L D, Hayes-Roth F, Lesser V R, Reddy D R. The HEARSAY-II Speech-Understanding System: Integrating Knowledge to Resolve Uncertainty[J]. ACM Computing Surveys, 1980, 12(2):213-253. <https://doi.org/10.1145/356810.356816>.
- [23] Nystrom R. Game Programming Patterns[M/OL]. Genever Benning, 2014[2026-05-22]. <https://gameprogrammingpatterns.com/>.

附 录

附录 A 项目主要文件清单

A.1 Godot 客户端核心文件

1. res://project.godot
2. res://levels/menu/MainMenu.tscn
3. res://levels/level_bunker_render.tscn
4. res://levels/outing/OutingMap.tscn
5. res://levels/outing/outing_map_level_v3.gd
6. res://characters/mirdo/mirdo_character.tscn

A.2 库存、状态与存档文件

1. res://scripts/Inventory/ItemData.gd
2. res://scripts/Inventory/inventory_storage_resource.gd
3. res://scripts/Inventory/shelter_inventory_resource.gd
4. res://scripts/character_resources/character_resource_state_component.gd

A.3 Mirdo AI 行为组件文件

1. res://scripts/character_ai/components/character_mind_state_component.gd
2. res://scripts/character_ai/components/character_perception_component.gd
3. res://scripts/character_ai/components/character_autonomous_planner_component.gd
4. res://scripts/character_ai/components/character_ai_action_executor_component.gd
5. res://scripts/character_ai/components/character_ai_dialogue_component.gd

A.4 后端 AI 服务文件

1. server://app/main.py
2. server://app/schemas.py
3. server://app/llm_provider.py
4. server://app/chat_orchestrator.py; server://app/expedition_orchestrator.py

致 谢

本设计与实现过程中，首先感谢指导教师在选题方向、系统设计和论文写作方面给予的指导。项目开发离不开 Godot、FastAPI、SQLite 等开源工具和相关文档资料，也感谢开源社区长期积累的教程与问题解答。角色字幕与 3D 字幕实现过程中，我参考了 B 站用户 StellarDreamerL（UID: 471766539）提供的 3D 字幕教程，该教程对我理解字幕在 3D 场景中的挂载、朝向和显示方式有很大帮助。也感谢在开发过程中参与测试和反馈的同学与朋友，他们提出的问题帮助我不断修正交互细节、存档流程和 AI 行为

表现。最后，感谢家人在毕业设计阶段给予的理解与支持。